



# Tiffin

## The Complete Beginner-to-Interview Guide

A real, ground-up course in every language this app uses — HTML, CSS, JavaScript, JSON, Python, Git and GitHub Actions, Capacitor — followed by a line-by-line walkthrough of the entire codebase, then an interview-ready Q&A guide.

**Prepared:** September 2026

**Audience:** Complete beginners with zero programming background

**How to use this document:** Part 1 is a genuine course — read it in order, work through the "check your understanding" questions at the end of each chapter before moving on, and don't skip sections even if they feel slow. Part 2 assumes you know everything in Part 1 and explains the real code. Part 4 is for interview prep once you're comfortable with the rest.

# Table of Contents

## PART 1

### Learn the Languages, From Zero

1. How Computers & the Web Work
2. HTML — Structure & Common Tags
3. HTML — Forms, Attributes & Accessibility
4. CSS — Selectors, Cascade & Specificity
5. CSS — The Box Model & Units
6. CSS — Colors
7. CSS — Flexbox
8. CSS — Grid
9. CSS — Positioning & Display
10. CSS — Responsive Design (Media Queries)
11. CSS — Transitions & Animations
12. CSS — Variables & Theming, In Depth
13. JavaScript — Your First Lines of Code
14. JavaScript — Variables & Primitive Types
15. JavaScript — Operators & Expressions
16. JavaScript — Control Flow (if/else/switch)
17. JavaScript — Loops
18. JavaScript — Functions, Deep Dive
19. JavaScript — Scope & Hoisting
20. JavaScript — Arrays, Deep Dive
21. JavaScript — Objects, Deep Dive
22. JavaScript — Strings & Basic Regex
23. JavaScript — Modern Syntax, In Depth
24. JavaScript — The DOM, In Depth
25. JavaScript — Events, In Depth
26. JavaScript — Closures & Event Delegation
27. JavaScript — Error Handling
28. JavaScript — Async JavaScript
29. JSON — The Data Format, In Depth
30. Browser Storage — localStorage, sessionStorage & Cookies
31. Browser & Web APIs
32. Python — From Zero
33. Python Applied — Reading build.py With Full Understanding
34. Git — Version Control Basics
35. GitHub Actions — CI/CD Basics
36. Capacitor — Turning a Website into a Phone App

## **PART 2**

### **The App, Explained End to End**

- 37. Project Tour — Every File and What It's For
- 38. The Build Pipeline — build.py Deep Dive
- 39. shell.html Deep Dive
- 40. app.css Deep Dive
- 41. food\_db.json Deep Dive
- 42. app.js Part 1 — Setup & The Shape of the Data
- 43. app.js Part 2 — Storage & Security
- 44. app.js Part 3 — Utility & Math Functions
- 45. app.js Part 4 — The Diary & Meal System
- 46. app.js Part 5 — Energy Calculations (BMR/TDEE)
- 47. app.js Part 6 — The Rendering System
- 48. app.js Part 7 — The Today View & Nutrients Tab
- 49. app.js Part 8 — The Food Picker Flow
- 50. app.js Part 9 — Custom Foods & Custom Meals
- 51. app.js Part 10 — Progress View & Charts
- 52. app.js Part 11 — Profile View
- 53. app.js Part 12 — Barcode Scanning
- 54. app.js Part 13 — Export / Import / Theme
- 55. app.js Part 14 — The Event System (the app's heart)
- 56. app.js Part 15 — Startup

## **PART 3**

### **Architecture & Design Decisions**

- 57. Why No Framework, No Backend?
- 58. Security: XSS, CSP, Prototype Pollution
- 59. The Full Data Flow, End to End

## **PART 4**

### **Interview Guide**

- 60. The 30-Second Pitch
- 61. Core Technical Q&A;
- 62. Deep-Dive Design Questions
- 63. Glossary — Every Term Used in This Guide

# PART 1

## Learn the Languages, From Zero

This part is a real course, not a glossary. Every concept here gets a plain-language explanation, a working code example, and — at the end of each chapter — a short set of questions so you can check you actually absorbed it before moving on. If a chapter feels slow, that's fine; Part 2 will lean on every single one of these ideas.

# 1. How Computers & the Web Work

## 1.1 What does it mean to "run code"?

A computer's processor can only do extremely simple things — add two numbers, compare two numbers, move a piece of data from one place to another — but it does billions of them per second. A **program** (or "code") is just a long, precise list of these simple instructions, written in a form a computer can follow. "Running" a program means the computer executes that list, instruction by instruction, top to bottom (skipping around when told to by loops and conditionals).

Humans don't write the raw instructions a processor understands directly — that's called **machine code**, and it's unreadable (streams of numbers). Instead, we write in a **programming language** — a human-readable set of rules for describing instructions — and a separate program (an **interpreter**, for languages like JavaScript and Python, or a **compiler**, for languages like C++) translates it into something the machine can actually run.

## 1.2 Files, folders, and extensions

Everything on a computer — including every piece of this project — is stored as a **file**: a named chunk of data saved to disk. Files live inside **folders** (also called **directories**), which can themselves contain more folders, forming a tree. A file's **extension** — the letters after the final dot in its name, like `.html` or `.js` — is just a convention telling humans and programs what *kind* of content that file holds; the computer doesn't magically know a file is JavaScript just because it's named `app.js` — a program (like a browser, or Node.js) has to choose to interpret it that way.

## 1.3 Client and server

Picture a restaurant. You (the **client**) don't go into the kitchen yourself — you ask a waiter for something, and the kitchen (the **server**) prepares it and sends it back. On the web, your browser is the client. When you type a web address, your browser sends a **request** out over the internet to a server — a computer, somewhere, that's always running and waiting for requests — asking for a specific file. The server sends back a **response**: usually an HTML file, plus whatever other files (CSS, JS, images) that HTML asks for.

This request/response conversation happens using a shared set of rules called **HTTP** (HyperText Transfer Protocol) — the "http://" (or the more secure, encrypted "https://") at the start of a web address names exactly this protocol.

**Note:** Tiffin is unusual precisely because, after the very first page load, it stops needing this conversation entirely — every file it needs (HTML, CSS, JS, even its whole food database) is downloaded once, and everything after that runs locally in your browser with no further requests. We'll see exactly how and why across this guide.

## 1.4 What the browser actually does with a web page

Once your browser has the HTML file, it works through a pipeline, roughly:

- **Parse HTML** — read the tags and build the DOM (Document Object Model), the tree of elements described in Chapter 2.
- **Parse CSS** — read all the style rules and build a matching tree of "what should each element look like."

- **Layout** — figure out the exact size and position, in pixels, of every element on the page.
- **Paint** — actually draw pixels to the screen based on that layout.
- **Run JavaScript** — execute any script, which can then read and modify the DOM, triggering the browser to redo layout and paint for whatever changed.

This entire pipeline can re-run, partially, many times a second — every time JavaScript changes something, or you resize the window, or you scroll. Keeping this fast is a large part of why the way you write CSS and JavaScript matters for a smooth-feeling page.

### Check your understanding — How the Web Works

1. What's the difference between a compiler and an interpreter?

*Answer: A compiler translates an entire program into machine code ahead of time, before it runs. An interpreter reads and executes a program's instructions directly (or just-in-time), without a separate translation step first. JavaScript and Python are both normally interpreted (with some just-in-time compilation happening behind the scenes for speed).*

2. In the client/server model, which one is your web browser?

*Answer: The client — it sends requests and displays whatever response it gets back.*

3. Name the five rough stages a browser goes through to show you a page.

*Answer: Parse HTML, parse CSS, layout, paint, and run JavaScript (which can then trigger layout/paint again).*

## 2. HTML — Structure & Common Tags

### 2.1 What HTML is, and isn't

HTML (HyperText Markup Language) is not a programming language — it has no math, no decisions, no loops. It's a **markup language**: you wrap text in tags to describe what that text is (a heading, a paragraph, a button), and the browser decides how to display each kind of thing.

### 2.2 Tags, elements, and nesting

```
<p>This is a paragraph.</p>

<div>
  <h2>A heading</h2>
  <p>A paragraph <strong>with some bold text</strong> inside it.</p>
</div>
```

A **tag** is the bracketed part, like `<p>`. An **element** is the tag plus everything between its opening and closing versions. Elements can contain other elements — this is called **nesting**, and it must be done in a strictly ordered way: whatever you open last, you must close first (like nested parentheses). The outer element here is `<div>`; `<h2>` and `<p>` are its **children**; `<strong>` is nested inside that `<p>`.

### 2.3 Block vs. inline elements

Every HTML element has a default "how does it take up space" behavior (which CSS can override, Chapter 9): **block** elements always start on a new line and stretch to fill the available width (like `<div>`, `<p>`, `<h1>`); **inline** elements sit within a line of text, only as wide as their content (like `<span>`, `<strong>`, `<a>`).

### 2.4 Void elements — tags with no closing tag

A handful of tags never wrap content, so they have no separate closing tag: `<br>` (line break), `<img>` (image), `<input>` (form field), `<meta>` (page metadata).

### 2.5 Lists and tables

```
<ul>                                <!-- unordered (bulleted) list -->
  <li>Rice</li>
  <li>Dal</li>
</ul>

<ol>                                <!-- ordered (numbered) list -->
  <li>Boil water</li>
  <li>Add rice</li>
</ol>

<table>
  <tr><th>Food</th><th>Calories</th></tr>
  <tr><td>Rice</td><td>200</td></tr>
</table>
```

### 2.6 Links and images

```
<a href="https://example.com">Click here</a>    <!-- href = where it links to -->
        <!-- src = file location; alt = fallback/screen-reader text -->
```

## 2.7 Semantic tags

Beyond generic `<div>`s, HTML offers tags that describe the *meaning* of a section, which helps both accessibility tools (like screen readers) and search engines understand a page's structure:

- `<header>` — introductory content, often navigation or a page title
- `<nav>` — a block of navigation links
- `<main>` — the primary content of the page
- `<section>` — a thematic grouping of content
- `<footer>` — closing content, like copyright or contact info

Tiffin's `shell.html` uses `<nav>` for its sidebar/bottom bar and `<main>` for its content area — a small but real example of this in practice (Chapter 20 in Part 2).

## 2.8 Comments

```
<!-- This text is ignored by the browser entirely; it's just a note for humans -->
```

### Check your understanding — HTML

1. What's the difference between a block element and an inline element?

*Answer: A block element starts on its own new line and stretches to fill the available width; an inline element flows within a line of text, taking up only as much width as its own content needs.*

2. Why does `<img>` have no closing tag, but `<div>` does?

*Answer: `<img>` is a 'void' element — it never wraps any content (the image itself is specified via the `src` attribute), so there's nothing for a closing tag to close. `<div>` is a container meant to wrap other content, so it needs an opening AND closing tag to mark where that content begins and ends.*

3. If you open `<div><p>`, in what order must you close them?

*Answer: Innermost first: `</p>` then `</div>` — nesting must close in the reverse order it opened, like matching parentheses.*

## 3. HTML — Forms, Attributes & Accessibility

### 3.1 Attributes, revisited

An attribute adds extra information to a tag, as `name="value"` pairs inside the opening tag. Some attributes work on almost any element (`id`, `class`, any `data-*` attribute); others are specific to one tag (`href` only makes sense on `<a>`, `src` only on things that load an external file like `<img>`).

### 3.2 Forms — collecting user input

A `<form>` groups related input fields so they can be handled — and, in a traditional server-driven website, submitted — together as one unit.

```
<form id="weightForm">
  <label for="weightInput">Weight (kg)</label>
  <input id="weightInput" type="number" name="weight" min="0" step="0.1" required>
  <button type="submit">Save</button>
</form>
```

`<label for="weightInput">` is linked to the input sharing that same `id` — clicking the label text then focuses that input automatically, and screen readers announce the label when the field receives focus. This is an accessibility best practice, not a decorative choice.

### 3.3 Common input types

| type     | Produces                                       |
|----------|------------------------------------------------|
| text     | A single-line free text field                  |
| number   | A numeric field, often with up/down steppers   |
| date     | A native date picker                           |
| checkbox | An on/off toggle                               |
| radio    | One choice among a group sharing the same name |
| file     | A button that opens the OS file picker         |
| password | Like text, but the typed characters are masked |

### 3.4 Validation attributes

HTML can enforce basic rules on its own, with zero JavaScript, before a form is even allowed to submit: `required` (must not be empty), `min`/`max` (numeric bounds), `minlength`/`maxlength` (text length bounds), `pattern` (a regex the value must match, see Chapter 22).

### 3.5 Buttons: `submit` vs. `button`

```
<button type="submit">Save</button>    <!-- triggers the enclosing form's submit event -->
<button type="button">Cancel</button>    <!-- does nothing on its own; needs a JS click handler -->
```

This distinction matters throughout Tiffin's code: buttons meant to submit a form use `type="submit"` (handled once, centrally, by the `submit` event, Chapter 25); buttons that should do something immediately on click, without any form involved, explicitly use `type="button"` so they never accidentally trigger a nearby form's submission.

### 3.6 Why accessibility matters here, concretely

Every icon-only button in this app (Chapter 2.8 in Part 2 will show the raw SVGs) also carries a `title` attribute describing what it does — this shows as a tooltip on hover for sighted mouse users, and gets read aloud by screen readers for users who can't see the icon at all.

#### Check your understanding — HTML Forms

1. What does `<label for="x">` do, and why bother?

*Answer: It links a label to the input with `id="x"`, so clicking the label text focuses that input, and screen readers announce the label's text when that input receives focus — a real accessibility requirement, not just a nicety.*

2. What's the practical difference between `<button type="submit">` and `<button type="button">`?

*Answer: `type="submit"` inside a `<form>` triggers that form's submit event automatically; `type="button"` does nothing by itself and needs an explicit JavaScript click handler to do anything.*

3. Name two form validation rules you can enforce with plain HTML, no JavaScript.

*Answer: Any two of: required (non-empty), min/max (numeric range), minlength/maxlength (text length), pattern (must match a given regular expression).*

## 4. CSS — Selectors, Cascade & Specificity

### 4.1 The rule shape, revisited

```
selector {  
  property: value;  
}
```

### 4.2 Selector types

| Selector           | Matches                                                                         |
|--------------------|---------------------------------------------------------------------------------|
| p                  | every <p> element                                                               |
| .btn               | every element with class="btn" (classes: reusable, many elements can share one) |
| #saveBtn           | the one element with id="saveBtn" (ids: must be unique per page)                |
| .card p            | every <p> INSIDE an element with class="card" (a 'descendant' selector)         |
| .card > p          | every <p> that is a DIRECT child of .card (not a grandchild)                    |
| input[type="text"] | every <input> whose type attribute equals "text"                                |
| .btn:hover         | a .btn element, only while the mouse hovers it                                  |
| .a, .b             | anything matching .a OR .b (comma = 'or')                                       |

### 4.3 The cascade — what happens when two rules conflict

"Cascading" in the name is literal: when multiple CSS rules could apply to the same element and set the same property, the browser has to pick a winner. It does so using three tie-breakers, in order:

- **Importance:** a declaration marked !important beats a normal one (used rarely — it's a blunt override that makes future overrides harder).
- **Specificity:** a more "specific" selector beats a less specific one — explained next.
- **Source order:** if importance and specificity are tied, whichever rule appears LATER in the stylesheet wins.

### 4.4 Specificity — how "specific" a selector is

Specificity is roughly calculated by counting: ID selectors count the most, then class/attribute/pseudo-class selectors, then plain tag-name selectors count the least.

```
p { color: black; }          /* specificity: 0-0-1  (one tag name) */  
.warning { color: red; }    /* specificity: 0-1-0  (one class) -- this wins over the rule above */  
#alert { color: orange; }   /* specificity: 1-0-0  (one id) -- this wins over both rules above */
```

This is why Tiffin's CSS relies mostly on classes (.btn-primary, .card) rather than IDs for styling — classes are reusable and have a moderate, predictable specificity, whereas ID-based rules are hard to

override later without resorting to even more specific (and messier) selectors.

## 4.5 Inheritance

Some CSS properties — mostly text-related ones like `color`, `font-family`, `font-size` — automatically pass down from a parent element to its children, unless a child rule overrides them. Other properties (like `border` or `padding`) do NOT inherit — giving a `<div>` a border doesn't give its children one too. This is exactly how Tiffin sets `font-family` once on `body` (app.css line 48) and every piece of text on the page inherits it without every single element needing its own `font-family` rule.

### Check your understanding — CSS Selectors & Cascade

1. Which wins: `.btn` or `#saveBtn`, if both set `color` on the same element?

*Answer: `#saveBtn` — ID selectors have higher specificity than class selectors, regardless of source order.*

2. What's the difference between `.card p` and `.card > p`?

*Answer: `'.card p'` matches any `<p>` anywhere inside `.card`, at any nesting depth. `'.card > p'` only matches a `<p>` that is a DIRECT child of `.card`, not a grandchild or deeper descendant.*

3. Does `border` inherit from a parent element to its children by default?

*Answer: No — border does not inherit. Color, font-family, and font-size are examples of properties that DO inherit by default; most box-model/layout properties (border, padding, margin, width) do not.*

## 5. CSS — The Box Model & Units

### 5.1 The box model, in detail

Every element is a rectangle made of four nested layers, from the inside out:

- **content** — the actual text/image/whatever the element holds
- **padding** — transparent space between the content and the border, still "inside" the element (its background color fills the padding too)
- **border** — a visible (or invisible) line drawn around the padding
- **margin** — transparent space OUTSIDE the border, separating this element from its neighbors (never filled by background color)

```
.card {  
  padding: 18px;    /* space inside, between border and content */  
  border: 1px solid #E7DAC0;  
  margin-bottom: 14px; /* space below, pushing the next element away */  
}
```

### 5.2 box-sizing — a crucial, easy-to-miss detail

By default, when you set `width: 200px`, that 200px applies only to the *content* — padding and border get added ON TOP, so the element's actual total width ends up larger than 200px. This surprises almost every beginner. The fix, used almost universally in modern CSS (and by this app, right at the top of `app.css`):

```
*, ::before, ::after {  
  box-sizing: border-box;  
}
```

`border-box` makes `width` include padding and border, so a 200px-wide element is ALWAYS 200px wide on screen, no matter how much padding/border you add — padding and border simply eat into the content area instead of adding extra width. The universal selector `*` matches every element on the page, so this one rule applies everywhere at once.

### 5.3 Units

| Unit    | Meaning                                                                      |
|---------|------------------------------------------------------------------------------|
| px      | pixels — an absolute, fixed size                                             |
| %       | a percentage of the PARENT element's corresponding size                      |
| em      | relative to the current element's own font-size                              |
| rem     | relative to the ROOT element's (<html>) font-size — more predictable than em |
| vw / vh | 1% of the viewport's (browser window's) width / height                       |

Tiffin mixes these deliberately: pixel values for things that should stay a fixed, precise size (border widths, icon sizes), percentages for fluid layout (`width: 100%` on inputs so they fill their container), and `vh` for

`min-height:100%`-style full-screen layouts.

## 5.4 display: none vs. visibility: hidden

Two different ways to hide an element, with an important difference: `display:none` removes it from the layout entirely — as if it weren't there at all, other elements shift to fill the gap. `visibility:hidden` makes it invisible but still reserves its space — other elements do NOT shift. Tiffin's responsive nav switch (Chapter 21 sidebar/bottomnav) uses `display:none`, since the hidden nav shouldn't leave an empty gap on the page.

### Check your understanding — The Box Model

1. With default box-sizing, if a `<div>` has `width:200px`, `padding:20px`, and `border:2px solid`, how wide does it actually render on screen?

*Answer: 244px (200 + 20+20 padding + 2+2 border) — with default content-box sizing, padding and border ADD to the specified width.*

2. Same question, but with box-sizing: border-box.

*Answer: Exactly 200px — border-box makes width include the padding and border, so they eat into the content area instead of adding extra size.*

3. You hide an element with `display:none`. Do the elements around it shift to fill the gap?

*Answer: Yes — display:none removes the element from the layout entirely, as if it isn't there. (visibility:hidden, by contrast, would hide it but still reserve its space.)*

## 6. CSS — Colors

### 6.1 Color formats

```
color: red; /* a small set of named colors */
color: #FF6B4A; /* hex: red, green, blue as 2-digit hex pairs, 00-FF each */
color: rgb(255, 107, 74); /* same color, as decimal red/green/blue, 0-255 each */
color: rgba(255, 107, 74, 0.5); /* like rgb, plus alpha (opacity): 0 = fully transparent, 1 = fully opaque */
color: hsl(11, 100%, 65%); /* hue (0-360 on a color wheel), saturation %, lightness % */
```

Tiffin's `app.css` uses hex colors for its named design tokens (`--accent: #FF6B4A`) but switches to `rgba` for things needing transparency, like `rgba(36, 28, 18, .45)` for the modal's semi-transparent dark backdrop — hex alone can't express partial transparency without a less common 8-digit hex form, so `rgba` is the clearer choice there.

### 6.2 `currentColor` — a color that reads from context

The special keyword `currentColor` refers to whatever that element's own `color` property currently is. This is how Tiffin's SVG icons (Chapter 23.2 in Part 2) automatically match whatever text color surrounds them — every icon's SVG markup uses `stroke="currentColor"` instead of a hardcoded color, so the exact same icon markup renders in gray in an inactive nav item and orange in an active one, purely by inheriting the surrounding `color` CSS property (Chapter 4.5's inheritance, applied cleverly).

#### Check your understanding — CSS Colors

1. Which color format lets you specify partial transparency: hex, rgb, or rgba?

*Answer: rgba — the fourth value (alpha) sets opacity from 0 (fully transparent) to 1 (fully opaque). Plain hex and rgb have no opacity channel.*

2. What does the CSS keyword `currentColor` do?

*Answer: It resolves to whatever that element's own `color` property is currently set to (inherited or explicit) — useful for things like SVG icons that should always match the surrounding text color without needing their own hardcoded color value.*

## 7. CSS — Flexbox

### 7.1 The mental model

Flexbox arranges a group of sibling elements along a single line — either a row or a column — and gives you powerful, simple control over how they share space, align, and wrap. You turn it on with `display: flex` on the PARENT; every property below then applies either to that parent (controlling its children as a group) or to an individual child.

```
.row {  
  display: flex;  
  flex-direction: row;          /* 'row' (default) or 'column' */  
  justify-content: space-between; /* spacing ALONG the main axis (row: horizontal) */  
  align-items: center;          /* alignment ACROSS the main axis (row: vertical) */  
  gap: 12px;                    /* space between every child */  
  flex-wrap: wrap;              /* allow children to wrap onto new lines if they don't fit */  
}
```

### 7.2 justify-content vs. align-items — the constant beginner mix-up

Both control alignment, but along different axes, and which axis is which depends on `flex-direction`. In the default row direction: `justify-content` controls horizontal spacing/alignment (the "main axis"), and `align-items` controls vertical alignment (the "cross axis"). Common values for `justify-content`: `flex-start`, `center`, `space-between` (push children to the far ends, evenly spacing any in between), `space-around`. Common values for `align-items`: `flex-start`, `center`, `stretch` (default — children stretch to fill the row's full height).

### 7.3 flex — controlling how one child grows or shrinks

```
.sidebar { flex: none; width: 216px; } /* never grow or shrink — stay exactly 216px */  
.main    { flex: 1; }                /* grow to consume all remaining space */
```

`flex: 1` is one of the most common flexbox declarations in existence — it means "take up any leftover space, sharing it equally with any other sibling that also has `flex: 1`." Tiffin's own top-level layout uses exactly this pattern (Chapter 21 in Part 2): the sidebar is fixed-width and `flex: none`, while the main content area is `flex: 1`, automatically filling whatever width the sidebar doesn't use.

#### Check your understanding — Flexbox

1. You set `display: flex` and `flex-direction: row` on a container. Which property controls horizontal alignment of its children — `justify-content` or `align-items`?

*Answer: justify-content — in the default row direction, the horizontal axis IS the main axis, which is exactly what justify-content controls. align-items would control the vertical (cross-axis) alignment in this same row layout.*

2. What does `flex: 1` on a child element mean?

*Answer: Grow to fill any remaining space in the flex container, sharing it equally with any sibling that also has flex: 1. It's commonly paired with a fixed-width sibling (flex: none) to make one panel fixed and another fill the rest.*

## 8. CSS — Grid

### 8.1 When to reach for Grid instead of Flexbox

Flexbox is fundamentally one-dimensional — one row, or one column. Grid is two-dimensional — rows AND columns at the same time — which makes it the right tool whenever you're laying out something that's genuinely grid-shaped, like a photo gallery or (in this app) a row of side-by-side stat cards.

```
.grid-2 {  
  display: grid;  
  grid-template-columns: 1fr 1fr; /* two columns, each taking an equal FRACTION of the space */  
  gap: 14px;  
}  
  
.mini-stat-grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr); /* shorthand for "1fr 1fr 1fr" -- three equal columns */  
  gap: 12px;  
}
```

### 8.2 The fr unit

fr ("fraction") is a unit that only exists in Grid: it means "one share of whatever space is left after fixed-size columns are accounted for." `grid-template-columns: 1fr 1fr` creates two equal-width columns; `200px 1fr` would create one fixed 200px column and one column that stretches to fill everything else — very similar in spirit to flexbox's `flex: 1`, but for a full 2D grid.

#### Check your understanding — Grid

1. What kind of layout problem calls for Grid over Flexbox?

*Answer: A genuinely two-dimensional layout — rows and columns at once, like a stat-card grid or photo gallery — as opposed to Flexbox's one-dimensional (single row OR column) model.*

2. What does `grid-template-columns: repeat(3, 1fr)` produce?

*Answer: Three equal-width columns, each taking one equal fraction (1fr) of the available space — shorthand for writing '1fr 1fr 1fr' by hand.*

## 9. CSS — Positioning & Display

### 9.1 The position property

| Value    | Behavior                                                                               |
|----------|----------------------------------------------------------------------------------------|
| static   | default — sits in normal document flow, top/left/etc. have no effect                   |
| relative | stays in normal flow, but top/left/etc. nudge it FROM where it'd normally be           |
| absolute | removed from flow; positioned relative to its nearest positioned ANCESTOR              |
| fixed    | removed from flow; positioned relative to the VIEWPORT — stays put even when scrolling |
| sticky   | normal flow until a scroll threshold, then behaves like fixed                          |

`position: absolute` only makes sense paired with a positioned ancestor — an element with `position: relative` (or `absolute`/`fixed`/`sticky`) somewhere above it in the tree, which becomes the reference point `absolute` positioning measures `top`/`left`/`etc.` from. Without one, it falls back to positioning relative to the entire page.

```
.ring-wrap { position: relative; width: 220px; height: 220px; }
.ring-center {
  position: absolute;
  inset: 0;           /* shorthand for top:0; right:0; bottom:0; left:0 -- fills the parent exactly */
}
```

This is exactly the pattern Tiffin uses to overlay the calorie number in the CENTER of the SVG progress ring (Chapter 29 in Part 2): the ring's wrapper is `position: relative`, and the text sitting on top of it is `position: absolute; inset: 0`, perfectly layering the number over the ring graphic underneath it.

Tiffin also uses `position: fixed` for its floating scan button and its toast notifications, and `position: sticky` for its top bar and sidebar — `sticky` is what lets the sidebar stay visible while its content area scrolls independently.

### 9.2 The display property

Every element has a `display` value controlling its fundamental layout behavior. We've already met `flex` and `grid`; the others worth knowing: `block`, `inline` (from Chapter 2.3), `inline-block` (inline flow, but you CAN set a width/height on it, unlike plain `inline`), and `none` (Chapter 5.4 — removed from layout entirely).

#### Check your understanding — Positioning & Display

1. An element has `position: absolute` but no ancestor has `position` set at all. What does it position itself relative to?

*Answer: The whole page (technically the initial containing block) — absolute positioning always needs a positioned ancestor (relative/absolute/fixed/sticky) to be useful as a LOCAL reference point; without one, it falls back to positioning against the page itself.*

2. What's the shorthand `inset: 0` equivalent to?

*Answer: `top: 0; right: 0; bottom: 0; left: 0` — commonly used to make an absolutely-positioned element exactly fill its positioned parent.*

## 10. CSS — Responsive Design (Media Queries)

### 10.1 Why responsive design matters

The same HTML/CSS has to look good on a phone screen (maybe 375px wide) and a desktop monitor (maybe 1920px wide) — a fixed-width, fixed-layout design would either be cramped on mobile or absurdly sparse on desktop. Responsive design means writing CSS that adapts based on the available space.

### 10.2 Media queries, revisited and expanded

```
.sidenav { width: 216px; }           /* desktop default */
.bottomnav { display: none; }       /* hidden by default */

@media (max-width: 860px) {
  .sidenav { display: none; }       /* hide the desktop sidebar ... */
  .bottomnav { display: flex; }     /* ... and show a bottom tab bar instead */
}

@media (min-width: 861px) {
  .fab-scan { bottom: 32px; right: 24px; } /* reposition a floating button on larger screens */
}
```

`max-width` means "apply this when the viewport is AT MOST this wide" (commonly used to target smaller screens); `min-width` means "apply this when the viewport is AT LEAST this wide" (commonly used to target larger screens). You can combine several conditions or write many separate `@media` blocks targeting different breakpoints — Tiffin deliberately keeps to just one breakpoint (860px) for simplicity, since its layout only really needs two states: "mobile" and "desktop."

### 10.3 Mobile-first vs. desktop-first

There are two philosophies for which styles are the "default" (outside any media query) and which are added inside one: **mobile-first** writes base styles for small screens, then uses `min-width` queries to add complexity for larger screens; **desktop-first** (what Tiffin does) writes base styles for the full desktop layout, then uses `max-width` queries to simplify/hide things for smaller screens. Neither is objectively correct — it's a matter of which is the more natural "starting point" for a given project.

#### Check your understanding — Responsive Design

1. What's the difference between `@media (max-width: 860px)` and `@media (min-width: 860px)`?

*Answer: `max-width:860px` applies its rules when the viewport is 860px wide OR NARROWER (commonly used to target smaller/mobile screens). `min-width:860px` applies when the viewport is 860px OR WIDER (commonly used to target larger/desktop screens).*

2. Does Tiffin use a mobile-first or desktop-first approach, and how can you tell?

*Answer: Desktop-first — its base (non-media-query) CSS describes the full sidebar/desktop layout, and a single `max-width:860px` query then simplifies it down to a mobile bottom-tab-bar layout.*

# 11. CSS — Transitions & Animations

## 11.1 Transitions — smoothly animating a property change

```
.btn {  
  background: var(--accent);  
  transition: transform .08s ease, background .15s ease;  
}  
.btn:active { transform: scale(.97); } /* transition makes this shrink SMOOTHLY, not instantly */
```

Without transition, a CSS property change (here, triggered by the `:active` pseudo-class from Chapter 3.6) would happen instantly, in a single frame — visually jarring. Adding `transition: property duration easing` tells the browser to automatically interpolate between the old and new values over that duration, producing a smooth animation with zero JavaScript.

## 11.2 Keyframe animations — for anything more complex than A-to-B

```
@keyframes fadeIn {  
  from { opacity: 0; transform: translateY(6px); }  
  to   { opacity: 1; transform: translateY(0); }  
}  
.fade-in { animation: fadeIn .35s ease both; }
```

A transition only animates between two states (before/after a change); a `@keyframes` animation can define any number of intermediate steps (from/to, or percentages like 0%, 50%, 100%) and, critically, can play automatically the moment an element appears — which is exactly what Tiffin's `.fade-in` class does every time a new view's HTML is inserted (Chapter 28 in Part 2), giving every screen transition a subtle, polished fade/slide-in instead of an abrupt pop.

## 11.3 Respecting reduced-motion preferences, revisited

Recall from earlier that `@media (prefers-reduced-motion: reduce)` lets motion-sensitive users disable these effects system-wide; this is a real, standard accessibility feature, not a nice-to-have.

### Check your understanding — Transitions & Animations

1. What's the key difference between a CSS transition and a `@keyframes` animation?

*Answer: A transition only smoothly interpolates between two states — a property's old value and its new value after some change (like a hover or click). A @keyframes animation can define any number of intermediate steps and can play automatically on its own, without needing a state change to trigger it.*

## 12. CSS — Variables & Theming, In Depth

### 12.1 Defining and using custom properties

Recall the syntax: define with two leading dashes inside a selector, read with `var(--name)`. Unlike a language variable, a CSS custom property is genuinely part of the cascade — it can be defined differently for different parts of the page, and child elements inherit whatever value is in effect for their nearest ancestor that set it.

```
:root {  
  --accent: #FF6B4A; /* the default, available everywhere */  
}  
.special-section {  
  --accent: purple; /* only elements INSIDE .special-section see this override */  
}
```

### 12.2 `var()` with a fallback

```
.thing { color: var(--maybe-undefined, black); } /* uses black if --maybe-undefined was never set */
```

### 12.3 Why this beats hardcoding colors everywhere

Without variables, supporting dark mode would mean writing two nearly-identical copies of every single color-related rule in the whole stylesheet — one for light, one for dark — and keeping them in sync forever as the design evolves. With variables, as Chapter 21 in Part 2 will show in full, dark mode is implemented by redefining a couple dozen variable values ONE time, in one place — every rule elsewhere in the file that reads `var(--text)` or `var(--accent)` updates automatically, with no per-rule duplication at all.

#### Check your understanding — CSS Variables

1. If `.special-section` redefines `--accent` to purple, does an element OUTSIDE `.special-section` see that new value?

*Answer: No — CSS custom properties follow the cascade and inheritance rules like any other property. Only elements INSIDE `.special-section` (that don't themselves override `--accent` again) see the purple value; everything outside still sees whatever `--accent` is set to at `:root` (or wherever else applies to them).*

2. Why does using CSS variables make supporting a dark theme drastically simpler than not using them?

*Answer: Because every rule in the stylesheet references the variable NAME (e.g. `var(--text)`) rather than a hardcoded value, switching themes only requires redefining what those variable names point to, once, in one place — not duplicating every color-related rule in the file for a second theme.*

## 13. JavaScript — Your First Lines of Code

### 13.1 What makes JavaScript different from HTML/CSS

HTML describes structure and CSS describes appearance, but neither can make a decision, do math, repeat an action, or respond to something happening. JavaScript is a full **programming language**: it has variables (memory), logic (decisions), and control flow (repetition, sequencing) — the same core ingredients every programming language has, just with its own particular syntax (rules for how code must be written).

### 13.2 Statements and semicolons

```
console.log("Hello, world!"); // prints text to the browser's developer console
let x = 5;                  // a statement that creates a variable
x = x + 1;                  // a statement that changes it
```

A **statement** is one complete instruction. JavaScript statements are conventionally ended with a semicolon (though the language will often — not always — infer one for you if you omit it; this codebase always writes them explicitly, which is the safer habit). `console.log(...)` is how a developer prints values out for debugging — it doesn't show up on the actual page, only in the browser's developer tools.

### 13.3 Comments

```
// a single-line comment -- everything after // on this line is ignored

/* a multi-line comment
   can span several lines */
```

Comments are notes for humans; the JavaScript engine skips them entirely. This app's source is heavily commented in exactly the places where the WHY isn't obvious from the code alone (Part 2 will point out several real examples) — good practice is commenting sparingly, only where the reasoning behind a piece of code genuinely needs explaining.

### 13.4 Where JavaScript code lives

In a browser, JavaScript can live inline inside a `<script>` tag, or in a separate `.js` file loaded via `<script src="app.js"></script>`. It runs top to bottom, in order, exactly once per page load (though it can register things — like event listeners — that run again later, in response to something happening).

#### Check your understanding — First Lines of Code

1. What's the difference between `console.log(...)` and something actually appearing on the web page?

*Answer: `console.log` prints a value only to the browser's developer console (a debugging tool) — it never appears anywhere on the visible page itself. To show something on the page, you have to modify the DOM directly (Chapter 24).*

2. Does the JavaScript engine execute the text inside `/* ... */`?

*Answer: No — that's a comment; everything between `/*` and `*/` (or after `//` on a line) is ignored entirely and exists purely for human readers.*

## 14. JavaScript — Variables & Primitive Types

### 14.1 const, let, and (avoid) var

```
const name = "Alice";    // cannot be reassigned -- attempting to will throw an error
let score = 100;         // CAN be reassigned
score = 150;             // fine

var old = "legacy";      // an older way to declare variables, with quirky scoping rules (Ch.19) --
                        // this codebase never uses it, and modern JS generally avoids it too
```

Best practice, followed throughout this codebase: default to `const`; only use `let` when a variable genuinely needs to be reassigned later. This isn't just style — it communicates intent to anyone reading the code, and lets the JavaScript engine (and your editor) catch a whole class of "I didn't mean to change that" bugs for free.

### 14.2 Primitive types, in depth

A **primitive** is a basic, non-object value. JavaScript has seven: number, string, boolean, null, undefined, symbol (rare, not used here), and bigint (rare, not used here). Everything else — arrays, objects, functions — is technically an **object** under the hood, covered in Chapters 20-21.

```
typeof 42;                // "number"
typeof "hi";              // "string"
typeof true;              // "boolean"
typeof undefined;         // "undefined"
typeof null;              // "object" -- a famous, long-standing quirk/bug in JS itself, kept for
                        // backwards compatibility; null is NOT actually an object
```

### 14.3 Type coercion — JavaScript's automatic conversions

JavaScript will sometimes silently convert one type to another to make an operation work, which is a common source of beginner confusion:

```
"5" + 3;                 // "53" -- + with a string on either side means CONCATENATE, so 3 becomes "3"
"5" - 3;                 // 2    -- but - only makes sense for numbers, so "5" becomes 5
"5" == 5;                // true  -- == converts types before comparing (avoid this operator, see below)
"5" === 5;               // false -- === requires the SAME type AND value, no conversion at all
NaN === NaN;             // false -- NaN ("Not a Number") is the only value in JS that isn't equal to itself!
Number.isNaN(NaN);       // true  -- the correct, safe way to actually check for NaN
```

**Note:** Always prefer `===` and `!==` over `==` and `!=`. The double-equals versions perform "loose" comparison with surprising coercion rules (`0 == ''` is true, `null == undefined` is true, but `null == 0` is false) that even experienced developers can't always predict from memory. This entire codebase uses `===` throughout.

### 14.4 Arrays and objects are NOT primitives

A quick preview before their dedicated chapters: an array (`[1, 2, 3]`) and an object (`{name: 'Alice'}`) both hold a **reference** to a location in memory, not the data itself directly the way a number or string does. This has a real, important consequence covered fully in Chapter 20: copying an array/object variable copies the reference, not the contents, so two variables can end up pointing at — and able to mutate — the exact same

underlying data.

### Check your understanding — Variables & Types

1. What does "5" + 3 evaluate to, and why?

*Answer: "53" — the string "5" concatenated with 3 (converted to the string "3"). Whenever + has a string on either side, JavaScript treats it as string concatenation rather than arithmetic.*

2. Why should you prefer === over == in this codebase (and generally)?

*Answer: == performs loose comparison, silently converting types before comparing, which produces surprising results (e.g. "" == 0 is true) that are easy to get wrong. === requires both the type and value to match exactly, with no hidden conversion, making code behavior far more predictable.*

3. Is const name = "Alice" the same restriction as "this string can never change"?

*Answer: Not quite — const means the VARIABLE BINDING can't be reassigned to point at a different value. For a primitive like a string this amounts to the same thing (strings are themselves immutable in JS), but for an array or object declared with const, the variable can't be reassigned to a different array/object, yet its CONTENTS can still be mutated (see Chapter 20).*

# 15. JavaScript — Operators & Expressions

## 15.1 Arithmetic operators

```
5 + 3;    // 8   addition
5 - 3;    // 2   subtraction
5 * 3;    // 15  multiplication
5 / 3;    // 1.6666... division
5 % 3;    // 2   remainder ("modulo") -- how much is left over after dividing as many whole times as possible
5 ** 2;   // 25  exponentiation (5 squared)
```

The remainder operator, %, comes up constantly in real code for things like "is this number even?" (`n % 2 === 0`) or cycling through a fixed list of options, which is exactly how Tiffin cycles a meal's emoji through its palette of choices (Chapter 32 in Part 2): `(i + 1) % EMOJI_PALETTE.length` wraps back around to 0 once it reaches the end of the list.

## 15.2 Comparison operators

| Operator                                | Meaning                                                    |
|-----------------------------------------|------------------------------------------------------------|
| <code>===</code>                        | strictly equal (same type AND value) -- always prefer this |
| <code>!==</code>                        | strictly NOT equal -- always prefer this                   |
| <code>==</code> / <code>!=</code>       | loosely equal/unequal, with type coercion -- avoid         |
| <code>&gt;</code> / <code>&lt;</code>   | greater than / less than                                   |
| <code>&gt;=</code> / <code>&lt;=</code> | greater than or equal / less than or equal                 |

## 15.3 Logical operators

```
true && false;    // false -- AND: both sides must be true
true || false;    // true  -- OR: at least one side must be true
!true;            // false -- NOT: flips a boolean

// short-circuit evaluation: && and || don't always evaluate BOTH sides
const a = user && user.name; // if `user` is falsy (e.g. null), stops there -- never tries
                             // to read .name on it, avoiding a crash
const b = savedName || 'Guest'; // if `savedName` is falsy/empty, falls back to 'Guest'
```

This "short-circuiting" behavior of `&&` and `||` is used as a control-flow shortcut constantly throughout this codebase — for example, `opts = opts || {}` ("if no options were passed, default to an empty object") and `dbFoodById(id) || null` (Chapter 7.2's earlier example), which relies on OR to supply a fallback the moment the first value is falsy.

## 15.4 The ternary operator, revisited

```
const label = isOver ? 'over' : 'under';

// nested (use sparingly -- readability suffers fast with more than one level):
const size = weight > 100 ? 'large' : weight > 50 ? 'medium' : 'small';
```

## 15.5 Assignment operators

```
let total = 0;
total += 5;    // same as: total = total + 5
total -= 2;    // same as: total = total - 2
total *= 3;    // same as: total = total * 3
```

### Check your understanding — Operators & Expressions

1. What does `7 % 3` evaluate to, and what does the `%` operator actually compute?

*Answer: 1 — % (modulo/remainder) gives you what's left over after dividing as many WHOLE times as possible: 3 goes into 7 twice (6), leaving a remainder of 1.*

2. In the expression ``user && user.name``, what happens if ``user`` is null?

*Answer: The expression short-circuits and evaluates to null (or whatever falsy value `user` was) WITHOUT ever attempting to read ``.name`` — this pattern is a common, safe way to avoid a crash from trying to access a property on something that might not exist.*

3. Rewrite ``total = total + 5`` using a shorthand assignment operator.

*Answer: `total += 5`*

## 16. JavaScript — Control Flow (if/else/switch)

### 16.1 if / else if / else, revisited with more detail

```
const age = 15;

if (age >= 18) {
  console.log('adult');
} else if (age >= 13) {
  console.log('teen');
} else {
  console.log('child');
}
// Only ONE branch ever runs -- the first condition (top to bottom) that evaluates truthy.
```

### 16.2 switch — choosing among many exact values

```
switch (action) {
  case 'nav':
    console.log('navigating');
    break;           // <-- without break, execution "falls through" into the NEXT case!
  case 'close-modal':
    console.log('closing');
    break;
  default:
    console.log('unknown action');
}
```

A switch is often used instead of a long if/else-if chain when you're comparing ONE value against many exact possibilities. Interestingly, this app deliberately does NOT use switch for its huge ~60-branch action dispatcher (Chapter 36 in Part 2) — it uses a long chain of `if (action === '...') { ...; return; }` statements instead, specifically because each branch there needs to return immediately afterward (skipping everything below), which an if-chain expresses more directly than a switch/break pattern would.

### 16.3 Truthy and falsy, revisited with the full list

Exactly six values are "falsy" in JavaScript — treated as false inside an if condition — and this list is worth memorizing completely, since EVERYTHING else, including an empty array/object, is truthy:

```
false, 0, -0, 0n, "", null, undefined, NaN  // ALL of JavaScript's falsy values

if ([]) { console.log('this runs!'); }      // an empty array is truthy -- easy to get wrong
if ({}) { console.log('this runs too!'); }  // an empty object is truthy too
```

#### Check your understanding — Control Flow

1. If you forget the `break` inside a switch case, what happens?

*Answer: Execution 'falls through' into the next case below it, running that code too, regardless of whether its own condition matches — a classic switch-statement bug.*

2. Is an empty array, `[]`, truthy or falsy in an if condition?

*Answer: Truthy — JavaScript's only falsy values are false, 0, -0, 0n, "" (empty string), null, undefined, and NaN. An empty array or empty object is NOT on that list, so both are truthy.*

# 17. JavaScript — Loops

## 17.1 The classic for loop

```
for (let i = 0; i < 5; i++) {  
  console.log(i); // prints 0, 1, 2, 3, 4  
}  
// the three parts: (1) run once before starting; (2) checked before EVERY iteration --  
// loop stops the moment this is false; (3) runs after EVERY iteration
```

## 17.2 while and do...while

```
let count = 0;  
while (count < 3) { // condition checked BEFORE each iteration -- may run zero times  
  console.log(count);  
  count++;  
}  
  
let n = 0;  
do {  
  console.log(n); // condition checked AFTER each iteration -- always runs at least once  
  n++;  
} while (n < 3);
```

## 17.3 for...of — the modern way to loop over an array's VALUES

```
const foods = ['rice', 'dal', 'roti'];  
for (const food of foods) {  
  console.log(food); // 'rice', then 'dal', then 'roti'  
}
```

## 17.4 for...in — looping over an object's KEYS

```
const goals = { calorieGoal: 2000, proteinGoal: 100 };  
for (const key in goals) {  
  console.log(key, goals[key]); // "calorieGoal 2000", then "proteinGoal 100"  
}
```

**Note:** In practice, this codebase almost never writes a raw for/while loop over an array or object — it uses array methods like `.map()`, `.forEach()`, and `.filter()` instead (Chapter 20), which is considered clearer, more modern style for array iteration specifically. Raw loops are still the right tool for things that AREN'T simply array iteration — a fixed number of repetitions, or looping backward through a range of dates, both of which this app's code does use a plain for loop for.

## 17.5 break and continue

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) break;      // exit the loop entirely, right now  
  if (i % 2 === 0) continue; // skip the REST of this iteration, jump to the next one  
  console.log(i);         // only prints odd numbers less than 5: 1, 3  
}
```

## Check your understanding — Loops

1. What's the key difference between while and do...while?

*Answer: while checks its condition BEFORE each iteration, so it might run zero times if the condition starts false. do...while checks its condition AFTER each iteration, so the loop body always runs at least once, even if the condition is false from the start.*

2. What's the difference between for...of and for...in?

*Answer: for...of iterates over an array's (or other iterable's) VALUES directly. for...in iterates over an object's (or array's) enumerable KEYS/indices — using for...in on an array is generally discouraged in favor of for...of or array methods.*

3. What does `continue` do differently from `break` inside a loop?

*Answer: continue skips the rest of the CURRENT iteration and jumps to the next one (the loop keeps going); break exits the loop entirely, immediately, with no further iterations at all.*

# 18. JavaScript — Functions, Deep Dive

## 18.1 Three ways to write a function

```
// 1. Function declaration -- "hoisted" (Ch.19): can be called even BEFORE this line runs
function add(a, b) {
  return a + b;
}

// 2. Function expression -- stored in a variable, NOT hoisted the same way
const subtract = function(a, b) {
  return a - b;
};

// 3. Arrow function -- shorter syntax, and handles `this` differently (see 18.4)
const multiply = (a, b) => {
  return a * b;
};
```

## 18.2 Parameters, arguments, and defaults

```
function greet(name = 'friend') { // 'friend' is used if no argument is passed at all
  return `Hello, ${name}`;
}
greet(); // "Hello, friend"
greet('Priya'); // "Hello, Priya"

function sum(...numbers) { // "rest parameter" -- gathers any number of arguments into an array
  return numbers.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3, 4); // 10
```

## 18.3 return — and what happens without one

A function without an explicit `return` statement implicitly returns `undefined`. This is a common source of bugs: calling a function purely for its side effects (like `saveState()`, which just writes to `localStorage`) is fine, but forgetting to `return` from a function you MEANT to produce a value from will silently hand back `undefined` instead of erroring.

## 18.4 this inside a regular function vs. an arrow function

`this` is a special keyword whose value depends on HOW a regular function is called (not where it's defined) — a notoriously confusing part of JavaScript. Arrow functions sidestep the confusion entirely: they don't have their own `this` at all — they simply use whatever `this` was in the surrounding code where they were written. This is a large part of why modern JavaScript (including every callback in this app's codebase) leans so heavily on arrow functions for short, inline callbacks.

## 18.5 Functions as values — the idea revisited

Because a function is just another kind of value, it can be stored in a variable, put inside an array or object, and — most importantly for this app — passed as an argument into another function, to be called later. This is

called a **callback**, and it's the entire mechanism behind event handling (Chapter 25) and array methods like `.map()` (Chapter 20).

### Check your understanding — Functions

1. What value does a function return if it has no explicit `return` statement?

*Answer: undefined — this is a silent, easy-to-miss bug source if you meant to compute and return something but forgot the return keyword.*

2. What's a `callback`?

*Answer: A function passed as an argument into another function, to be called (invoked) by that other function later — the basis of how array methods like `.map()` and event listeners both work.*

3. Why do arrow functions avoid a common source of `this`-related bugs that regular functions have?

*Answer: Arrow functions don't define their own `this` at all — they inherit whatever `this` was in the surrounding code where they were written, rather than having it depend on how/where the function gets CALLED, which is the confusing behavior regular functions have.*

# 19. JavaScript — Scope & Hoisting

## 19.1 What "scope" means

Scope is the answer to "from where in the code can I see this variable?" JavaScript has three kinds:

- **Global scope** — declared outside any function or block; visible EVERYWHERE in the file (the reason Tiffin wraps its entire app in an IIFE, Chapter 26, to avoid polluting this).
- **Function scope** — a variable declared inside a function is only visible inside that function (and any functions nested within it).
- **Block scope** — a variable declared with `const/let` inside a `{ }` block (an if, a loop, even a bare pair of braces) is only visible inside that block.

```
function example() {  
  if (true) {  
    const x = 5;    // block-scoped -- only exists inside this if-block  
    let y = 10;     // also block-scoped  
    var z = 15;     // FUNCTION-scoped, not block-scoped -- leaks out of the if-block!  
  }  
  console.log(z);  // 15 -- var "leaked" out of the block  
  console.log(x);  // ReferenceError -- x doesn't exist out here  
}
```

This is a major reason modern style avoids `var`: its function-level (rather than block-level) scoping is a frequent source of confusing bugs, especially inside loops. `const` and `let` were added to the language specifically to provide predictable, block-level scoping instead.

## 19.2 The scope chain

When code references a variable, JavaScript looks for it in the current (innermost) scope first; if not found there, it looks in the next scope out, and so on, all the way to the global scope — this search path is called the **scope chain**. A function defined inside another function can always see ("close over") the outer function's variables — this is the foundation of closures, covered fully in Chapter 26.

## 19.3 Hoisting

JavaScript processes a script in two conceptual passes: first it scans for declarations and sets up memory for them, THEN it runs the code top to bottom. This means function declarations (not expressions) are fully usable even before the line where they're written — this is called **hoisting**.

```
sayHi();           // works! "Hi" is printed -- function declarations are hoisted entirely  
  
function sayHi() {  
  console.log('Hi');  
}  
  
console.log(mystery); // ReferenceError -- let/const are hoisted too, but left in an unusable  
let mystery = 5;      // "temporal dead zone" until their actual declaration line runs
```

Because of this, this app's code always defines its functions before an event handler needs to call them — relying on hoisting for top-level function declarations is normal, expected JavaScript style, but relying on it for

let/const variables would simply crash, so those are always declared before their first use.

### Check your understanding — Scope & Hoisting

1. A variable declared with `var` inside an if-block — is it visible outside that block, in the same function?

*Answer: Yes — var is function-scoped, not block-scoped, so it 'leaks' out of if/for/while blocks (though still not out of the enclosing function). const and let, by contrast, are genuinely block-scoped and would NOT be visible outside that same if-block.*

2. Can you call a function declared with `function name() {...}` from a line of code ABOVE where it's written in the file?

*Answer: Yes — function declarations (not function expressions or arrow functions assigned to a variable) are fully hoisted, meaning the whole function is available throughout its scope even before the line that defines it actually executes.*

3. What is the 'scope chain'?

*Answer: The ordered sequence of scopes JavaScript searches through to resolve a variable reference: current (innermost) scope first, then each enclosing scope outward, ending at the global scope. A nested function can always see variables from any scope that encloses it.*

## 20. JavaScript — Arrays, Deep Dive

### 20.1 Creating and accessing arrays

```
const fruits = ['apple', 'banana', 'cherry'];
fruits[0];           // 'apple' -- indexes start at 0
fruits[fruits.length - 1]; // 'cherry' -- the last item, using .length
fruits.length;       // 3
```

### 20.2 Arrays hold REFERENCES, not copies

```
const a = [1, 2, 3];
const b = a;           // b now points to the SAME array in memory, not a new copy
b.push(4);
console.log(a);        // [1, 2, 3, 4] -- a changed too! They're the same array.

const c = [...a];      // spread copies the array's CURRENT items into a brand new array
c.push(5);
console.log(a);        // still [1, 2, 3, 4] -- c is independent now
```

This is exactly why Tiffin writes `[...STATE.weightLog].sort(...)` rather than `STATE.weightLog.sort(...)` directly (Chapter 27, 32 in Part 2) — `.sort()` mutates the array it's called on IN PLACE, and sorting a plain reference to the real state array would silently reorder the actual stored data as a side effect of what's supposed to be a read-only "give me this sorted for display" operation.

### 20.3 Mutating vs. non-mutating methods — a crucial distinction

| Mutating (changes original) | Non-mutating (returns a new array) |
|-----------------------------|------------------------------------|
| push, pop, shift, unshift   | map, filter, slice                 |
| splice, sort, reverse       | concat, join                       |

```
const arr = [1, 2, 3];
arr.push(4);           // arr is now [1, 2, 3, 4] -- MUTATED in place; push returns the new length, 4

const doubled = arr.map(n => n * 2); // returns a BRAND NEW array: [2, 4, 6, 8]
console.log(arr);       // arr itself is unchanged: [1, 2, 3, 4]
```

### 20.4 The core methods used throughout this app

#### `.map()` — transform every item into something new, 1-to-1

```
const kcals = foods.map(food => food.kcal);
const html = foods.map(f => `<div>${f.name}</div>`).join(''); // build HTML strings from data
```

#### `.filter()` — keep only items that pass a test

```
const indianFoods = foods.filter(f => f.category === 'Indian');
```

## **.forEach()** — do something with every item, without producing a new array

```
NUTRIENT_KEYS.forEach(key => {
  total[key] = (a[key] || 0) + (b[key] || 0);
});
```

## **.find()** — the first matching item, or undefined

```
function dbFoodById(id) {
  return FOOD_DB.find(f => f.id === id) || null;
}
```

## **.reduce()** — combine every item down into a single value

```
const total = [10, 20, 30].reduce((sum, n) => sum + n, 0); // 60
// (sum, n) => ... runs once per item; `sum` carries forward between calls, starting at 0
```

## **Other frequently useful methods**

```
arr.includes(3);           // true if 3 is anywhere in the array
arr.indexOf(3);           // the index of the first 3, or -1 if not found
arr.some(n => n > 2);       // true if AT LEAST ONE item passes the test
arr.every(n => n > 0);      // true only if ALL items pass the test
arr.join(', ');           // combine into one string: "1, 2, 3"
arr.slice(1, 3);          // a NEW array containing items at index 1 up to (not including) 3
arr.splice(1, 1);         // MUTATES: removes 1 item starting at index 1, returns the removed items
```

**Note:** `.slice()` and `.splice()` look alike but behave very differently: `slice` is non-mutating (safe to use on data you don't want to change) and `splice` mutates in place. This app uses `.splice()` deliberately whenever it wants to remove an entry from the diary (e.g. deleting a logged food) — mutating STATE directly is exactly the intended behavior there.

## **Check your understanding — Arrays**

1. `const a = [1,2]; const b = a; b.push(3);` — does ``a`` now include the 3 as well?

*Answer: Yes — ``b = a`` copies the REFERENCE to the array, not the array's contents, so ``a`` and ``b`` point to the exact same array in memory. Mutating one mutates the other. To get an independent copy you'd need something like ``const b = [...a]``.*

2. Which of these mutate their array in place: `.map()`, `.push()`, `.filter()`, `.splice()`?

*Answer: `.push()` and `.splice()` mutate in place. `.map()` and `.filter()` do NOT mutate — they return a brand new array, leaving the original untouched.*

3. What does `.find()` return if nothing in the array matches?

*Answer: `undefined` — which is why this codebase commonly writes ``find(...)`` `|| null`` to turn that into an explicit, intentional null instead.*

# 21. JavaScript — Objects, Deep Dive

## 21.1 Object literals, revisited

```
const food = {  
  name: 'Rice',  
  kcal: 200,  
  isVegetarian: true,  
};
```

## 21.2 Dot notation vs. bracket notation

```
food.name;           // 'Rice' -- dot notation: the key name is written literally in the code  
food['name'];        // 'Rice' -- bracket notation: same result  
  
const key = 'kcal';  
food[key];           // 200 -- bracket notation lets the KEY itself be a variable/expression  
food.key;            // undefined -- this looks for a property LITERALLY named "key", which doesn't exist!
```

Bracket notation with a variable is essential whenever a property name isn't known until the code runs. Recall Chapter 20's `NUTRIENT_KEYS` array of nutrient name strings — every nutrient-math function in Tiffin loops over that array and reads `food[key]` for each one, which is only possible with bracket notation (dot notation could never express "look up whatever key this loop variable currently holds").

## 21.3 Objects also hold references

```
const original = { x: 1 };  
const alias = original;  
alias.x = 99;  
console.log(original.x); // 99 -- same underlying object, same as arrays in Ch.20.2  
  
const copy = { ...original }; // spread makes a shallow copy -- a NEW, independent object  
copy.x = 5;  
console.log(original.x);     // still 99 -- copy is independent
```

"Shallow" copy matters: `{...original}` copies `original`'s own top-level keys into a new object, but if one of those values is ITSELF an object or array, that nested value is still just a reference, shared between the original and the copy. Tiffin's `{...DEFAULT_GOALS}` pattern (Chapter 23 in Part 2) works safely here specifically because every value inside `DEFAULT_GOALS` is a primitive (a plain number or string), not a nested object — so a shallow copy is all that's needed to get a fully independent goals object per user.

## 21.4 Useful Object methods

```
Object.keys({a:1, b:2}); // ['a', 'b'] -- an array of just the keys  
Object.values({a:1, b:2}); // [1, 2] -- an array of just the values  
Object.entries({a:1, b:2}); // [['a',1], ['b',2]] -- an array of [key, value] pairs  
Object.assign({}, obj1, obj2); // merges obj2's keys ONTO A COPY of obj1 (obj1 itself untouched)
```

## 21.5 `hasOwnProperty` — checking if a key genuinely exists

```
const obj = { name: 'Alice' };
'name' in obj; // true
obj.hasOwnProperty('name'); // true -- checks the object's OWN keys specifically
Object.prototype.hasOwnProperty.call(obj, 'name'); // true, but safer -- doesn't rely on `obj`
// still having the normal Object prototype
```

That last, more defensive form appears verbatim in this app's `mergeKnownFields()` function (Chapter 24 in Part 2) — a real, deliberate security choice, not just a stylistic preference, explained fully there.

### Check your understanding — Objects

1. Given `const key = 'age'; const person = {age: 30};` — how do you read the age using the `key` variable?

*Answer: `person[key]` — bracket notation lets the property name be computed/dynamic. `person.key` would instead look for a literal property named "key", which doesn't exist here.*

2. Does `{...original}` produce a fully independent, deep copy of a nested object?

*Answer: No — spread produces a SHALLOW copy: the object's own top-level keys are copied into a new object, but if any value is itself an object/array, that nested value is still a shared reference between the original and the copy.*

3. What's the difference between `Object.keys(obj)` and `Object.entries(obj)`?

*Answer: `Object.keys` returns an array of just the property names. `Object.entries` returns an array of `[key, value]` pairs for every property — strictly more information.*

## 22. JavaScript — Strings & Basic Regex

### 22.1 Strings, revisited

```
const s = "Hello, World!";
s.length;           // 13
s.toUpperCase();     // "HELLO, WORLD!"
s.toLowerCase();     // "hello, world!"
s.trim();            // removes whitespace from both ends
s.includes("World"); // true
s.slice(0, 5);        // "Hello" -- like array slice, works on characters
s.replace("World", "There"); // "Hello, There!" -- replaces the FIRST match only
s.split(", ");        // ["Hello", "World!"] -- turns a string into an array
```

### 22.2 Template literals, revisited in full

```
const name = "chicken curry", kcal = 350;
const msg = `Added ${name} (${kcal} kcal)`; // embeds expressions directly

// can contain ANY expression, not just a variable name:
const status = `${kcal > 300 ? 'high' : 'normal'} calorie`;

// can span multiple lines -- exactly how this app builds multi-line HTML fragments:
const html = `

### 22.3 Regular expressions (regex) — pattern matching in text



A regex is a compact, specialized mini-language for describing a PATTERN a string might match, rather than one exact literal string. Full regex syntax is a topic of its own, but a small amount goes a long way, and this app uses exactly one regex, worth understanding completely:



```
const pattern = /[<>"']/g;
//           ■       ■
//           ■       ■■ 'g' flag: find ALL matches, not just the first
//           ■■■■■■■■ a character class: matches ANY ONE of these 5 characters

"Tom & Jerry".replace(pattern, c => '[' + c + ']');
// -> "Tom [&] Jerry"
```



[...] defines a character class — match any single character that appears inside the brackets. [<>"' ] matches any one of & < > " ' — precisely the five characters that are dangerous to leave unescaped inside HTML. The g ("global") flag after the closing slash means "don't stop at the first match — replace EVERY occurrence in the whole string." This exact regex is the heart of Tiffin's escapeHtml() function, covered fully in Chapter 39 of Part 2.


```

```
function escapeHtml(s) {  
  return String(s).replace(/[<>"]/g, c => ({  
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'  
  }[c]));  
}
```

`.replace(pattern, callback)` — when the second argument is a function instead of a plain replacement string, that function gets called once PER MATCH, receiving the matched character, and whatever it returns becomes the replacement. Here, the callback looks up each dangerous character in a small object and returns its safe HTML-entity equivalent.

### Check your understanding — Strings & Regex

1. What does `"Hello".slice(0, 3)` return?

*Answer: "Hel" — `slice(start, end)` returns characters from index `start` up to (but not including) index `end`, just like array slicing.*

2. In the regex `/[<>"]/g`, what does the trailing `g` flag do?

*Answer: It makes the match/replace operate globally — finding and acting on EVERY occurrence in the string, not just stopping after the first one.*

3. Why does `escapeHtml`'s `.replace()` call pass a FUNCTION as its second argument instead of a plain string?

*Answer: Because the correct replacement is different depending on WHICH dangerous character was matched (& needs &, < needs <, etc.) — a function gets called once per match with that exact character, letting it look up and return the right replacement each time, rather than replacing every match with one single fixed string.*

## 23. JavaScript — Modern Syntax, In Depth

### 23.1 Destructuring, revisited with more forms

```
// Array destructuring
const [first, second] = ['a', 'b'];

// Skipping items
const [, , third] = ['a', 'b', 'c']; // third = 'c'

// Object destructuring
const { name, age } = person;

// Renaming while destructuring
const { name: userName } = person; // userName now holds person.name

// Default values if the property is missing
const { theme = 'system' } = settings;

// Nested destructuring
const { profile: { sex, age: userAge } } = STATE;
```

### 23.2 Rest and spread, the full picture

The same `...` syntax means two different (opposite) things depending on context: **spread** EXPANDS a collection outward; **rest** GATHERS remaining items inward.

```
// SPREAD -- expanding
const combined = [...arr1, ...arr2]; // one new array containing both arrays' items
const merged = { ...defaults, ...overrides }; // one new object, overrides' keys win on conflict

// REST -- gathering (opposite direction)
function sum(...numbers) { /* numbers is an array of every argument passed in */ }
const [head, ...tail] = [1, 2, 3, 4]; // head = 1, tail = [2, 3, 4]
const { kcal, ...otherNutrients } = food; // kcal pulled out separately, everything else grouped
```

### 23.3 Optional chaining (?.) and nullish coalescing (??)

```
const city = user?.address?.city;
// If `user` is null/undefined, OR `user.address` is null/undefined, this short-circuits
// to `undefined` immediately, instead of throwing "Cannot read property 'city' of undefined"

const goal = settings.calorieGoal ?? 2000;
// ?? only falls back if the left side is null or undefined SPECIFICALLY --
// unlike ||, it does NOT treat 0 or '' as "missing"
const goalOr = settings.calorieGoal || 2000;
// with ||, a legitimately-set calorieGoal of 0 would ALSO trigger the fallback to 2000 -- a real bug!
```

The distinction between `||` and `??` matters a lot for exactly the kind of numeric settings this app stores — a goal of `0` is a valid (if unusual) value a user might genuinely set, and `||` would incorrectly treat it as "missing" and silently override it, whereas `??` only treats `null/undefined` specifically as "missing."

[Check your understanding — Modern Syntax](#)

1. What's the difference between spread and rest, given they use the identical ... syntax?

*Answer: Spread expands an existing array/object OUT into individual elements/keys (e.g. building a new array from pieces of others). Rest does the opposite — it GATHERS multiple individual values INTO one array (e.g. collecting a function's extra arguments, or the 'remaining' items after destructuring some off the front).*

2. Why might `settings.calorieGoal || 2000` be a bug if calorieGoal is legitimately set to 0?

*Answer: Because 0 is falsy, || would treat a genuinely-set value of 0 as if it were missing and incorrectly fall back to 2000. The nullish coalescing operator, ??, avoids this — it only falls back when the left side is specifically null or undefined, treating 0 (and "") as valid, present values.*

3. What does `user?.address?.city` evaluate to if `user.address` doesn't exist?

*Answer: undefined, safely — optional chaining short-circuits the moment it hits a null/undefined link in the chain, rather than throwing an error trying to read a property off of undefined.*

## 24. JavaScript — The DOM, In Depth

### 24.1 The DOM tree, revisited

Recall from Part 1's earlier chapter that the DOM is the browser's live, in-memory tree of every element on the page. Each element is a JavaScript object with properties and methods you can read and call directly.

### 24.2 Every way to select an element

```
document.getElementById('viewRoot');      // the ONE element with this exact id
document.querySelector('.food-row');      // the FIRST element matching this CSS selector
document.querySelectorAll('.food-row');   // ALL matches, as a NodeList (array-like, not a real array)
document.querySelector('[data-action="nav"]'); // CSS attribute selectors work here too
```

### 24.3 Traversing the tree from a known element

```
el.parentElement;      // the element directly containing this one
el.children;           // this element's direct child elements (a live HTMLCollection)
el.closest('.card');    // walks UP through ancestors until one matches, or returns null
el.querySelector('.icon'); // searches DOWN, only within this element's own descendants
```

`.closest()` is the single most-used traversal method in this app's entire codebase — it's the mechanism the whole event delegation system (Chapter 27) relies on to find "which actionable element, if any, was this click inside of."

### 24.4 Reading and changing content

```
el.textContent;      // reads/sets PLAIN TEXT only -- safe, never parses HTML
el.textContent = 'Hello';

el.innerHTML;        // reads/sets HTML -- browser PARSES whatever string you assign
el.innerHTML = '<b>Hi</b>'; // creates a real, bold <b> element -- powerful, but risky with
                           // untrusted text (Chapter 39's XSS discussion)
```

### 24.5 Attributes, classes, and inline styles

```
el.getAttribute('data-id');      // read any attribute's value
el.setAttribute('data-id', '123'); // set any attribute
el.dataset.id;                   // shortcut specifically for data-* attributes: reads data-id
el.dataset.action;               // reads data-action

el.classList.add('active');
el.classList.remove('active');
el.classList.toggle('active');    // adds it if missing, removes it if present
el.classList.contains('active');  // true/false

el.style.width = '50%';           // sets ONE inline CSS property directly
```

### 24.6 Creating and inserting new elements

```
const div = document.createElement('div');
div.className = 'toast';
div.textContent = 'Saved!';
document.body.appendChild(div);           // adds it as the LAST child of <body>

el.remove();                             // removes an element from the page entirely
```

**Note:** Tiffin's toast notifications (Chapter 24 in Part 2) are one of the few places this app builds an element with `createElement + appendChild` rather than an `innerHTML` string — almost everything else in the app is built as one big HTML string and inserted in a single `.innerHTML =` assignment instead, which is simpler when you're regenerating a whole section at once rather than adding one small element.

### Check your understanding — The DOM

1. What's the difference between `el.textContent = '...'` and `el.innerHTML = '...'`?

*Answer: `textContent` always treats the string as PLAIN TEXT, with no HTML parsing at all — safe by construction. `innerHTML` PARSES the string as HTML, creating real elements from any tags inside it — powerful, but risky if the string contains untrusted user data (see Chapter 39's XSS discussion).*

2. What does `el.closest('.card')` do?

*Answer: Starting from `el`, it walks UPWARD through ancestor elements (`el` itself, then its parent, then its parent's parent, etc.) until it finds one matching the given selector, or returns `null` if none match all the way to the document root.*

3. How do you read the value of a `data-action="delete"` attribute in JavaScript?

*Answer: `el.dataset.action` — the ``dataset`` property is a shortcut specifically for reading/writing any `data-*` attribute, converting the dash-separated name (`data-action`) into a camelCase property (`dataset.action`).*

## 25. JavaScript — Events, In Depth

### 25.1 Listening for events, revisited

```
button.addEventListener('click', function(event) {
  console.log('clicked!', event.target);
});

button.removeEventListener('click', handlerFn); // requires the SAME function reference to remove it
```

### 25.2 Common event types

| Event            | Fires when...                                                             |
|------------------|---------------------------------------------------------------------------|
| click            | an element is clicked (mouse or, on touch devices, tapped)                |
| input            | a text field's value changes, on EVERY keystroke                          |
| change           | a field's value is committed -- e.g. a file picker, dropdown, or checkbox |
| submit           | a <form> is submitted (button click OR pressing Enter in a field)         |
| keydown / keyup  | a key is pressed down / released                                          |
| DOMContentLoaded | the browser has finished parsing the initial HTML                         |
| load             | a resource (image, whole page, etc.) has fully finished loading           |

### 25.3 The event object

Every handler receives one argument — the event object — carrying details about what happened:

```
function onClick(e) {
  e.target;           // the EXACT element that was clicked (could be deep inside a button, e.g. an icon)
  e.currentTarget;    // the element the LISTENER was actually attached to
  e.type;             // 'click'
  e.preventDefault(); // cancel the browser's default behavior for this event
  e.stopPropagation(); // stop this event from bubbling up to ancestor elements
}
```

### 25.4 e.target vs. e.currentTarget — a frequent point of confusion

If you click an <svg> icon sitting inside a <button>, and the listener is attached to document (as in this app's event delegation pattern, Chapter 27): `e.target` is the SVG itself (the innermost element actually clicked); `e.currentTarget` is document (where the listener lives). This is exactly why Tiffin's click handler uses `e.target.closest('[data-action]')` rather than `e.target` alone — clicking the icon needs to resolve back up to the actionable button that contains it.

### 25.5 preventDefault vs. stopPropagation — not the same thing

These are commonly confused because both sound like "stop this event." They stop two DIFFERENT things: `preventDefault()` cancels the BROWSER's own built-in behavior for this event (like a form actually submitting/reloading the page, or a link actually navigating) — the event still bubbles up normally afterward. `stopPropagation()` stops the event from bubbling any further UP through ancestor elements — it has nothing to do with the browser's default behavior at all. A handler can call either, both, or neither, depending on what it needs.

## 25.6 Event bubbling and capturing

Recall from earlier: a DOM event fires first on the exact element clicked, then "bubbles" upward through every ancestor, all the way to `document`. (There's also a rarer "capturing" phase that runs top-down BEFORE bubbling, opted into with a third `addEventListener` argument — not used anywhere in this app.) Bubbling is the entire foundation event delegation (Chapter 27) is built on.

### Check your understanding — Events

1. A click lands on an icon inside a `.svg`. If a listener is attached to `document`, what is `e.target` -- the `svg`, the button, or `document`?

*Answer: The `svg` — `e.target` is always the exact, innermost element the event actually originated on, regardless of where the listener itself is attached.*

2. What's the difference between `event.preventDefault()` and `event.stopPropagation()`?

*Answer: `preventDefault()` cancels the browser's own built-in behavior for that event (like a form actually submitting, or a link actually navigating) — the event still bubbles normally. `stopPropagation()` stops the event from bubbling further up to ancestor elements — it has no effect on the browser's default behavior.*

3. What event fires on every keystroke inside a text input: `input` or `change`?

*Answer: `input` — it fires immediately on every keystroke/value change. `change` fires later, when the value is COMMITTED (e.g. the field loses focus, or — for things like file pickers/checkboxes — the moment a discrete choice is made).*

## 26. JavaScript — Closures & Event Delegation

### 26.1 Closures, revisited in depth

A closure is what you get whenever a function is defined inside another function: the inner function permanently "remembers" — closes over — the variables that were in scope around it, even after the outer function has already finished running and would normally have its local variables cleaned up.

```
function makeCounter() {
  let count = 0; // private to this specific call of makeCounter
  return function() {
    count = count + 1; // this inner function "closes over" `count`
    return count;
  };
}

const counterA = makeCounter();
const counterB = makeCounter(); // a SEPARATE, independent `count` for this call
counterA(); // 1
counterA(); // 2
counterB(); // 1 -- counterA and counterB don't share state; each closure has its own `count`
```

Closures are how `picker`, `mealBuilder`, and every other piece of "currently open modal's working state" in Tiffin's codebase stays alive and accessible across many separate event-handler calls, without needing to be stored on the DOM itself — they're just variables declared once, near the top of the IIFE (Chapter 19.1's global-within-the-IIFE scope), that every handler function can see and mutate.

### 26.2 IIFEs, revisited

```
(function(){
  "use strict";
  // every const/let declared in here is invisible OUTSIDE this function
})(); // <-- the trailing () calls the function immediately, right where it's defined
```

This is the exact top-level wrapper around Tiffin's entire `app.js`. Because the function is anonymous and called immediately, it never leaves a named function lying around in global scope either — the whole ~1,700-line app contributes exactly nothing to the global namespace.

### 26.3 Event delegation, the complete explanation

We've now covered every prerequisite: event bubbling (Ch.25.6), `e.target` (Ch.25.4), `.closest()` (Ch.24.3), and `data-*` attributes (Ch.3.1, Ch.24.5). Event delegation combines all four into one pattern:

```
document.addEventListener('click', function(e) {
  const el = e.target.closest('[data-action]');
  if (!el) return; // click landed somewhere with no action -- ignore it
  const action = el.dataset.action;

  if (action === 'delete-entry') { /* ... */ return; }
  if (action === 'add-water') { /* ... */ return; }
  // ...dozens more
});
```

Instead of attaching a separate listener to every one of potentially hundreds of buttons across every screen, ONE listener on `document` catches every click, anywhere on the page, forever — including on buttons that don't exist yet at page-load time and get created later by a render function. This last point is the biggest practical win: since Tiffin constantly replaces large chunks of the DOM via `innerHTML` (Chapter 28 in Part 2), any per-element listener would be destroyed along with the old HTML and would need to be manually re-attached after every single re-render. A delegated listener on `document` never needs re-attaching, because it was never attached to the elements that keep getting replaced in the first place.

### Check your understanding — Closures & Event Delegation

1. In the `makeCounter()` example, why does calling `counterB()` return 1, not 3, even after `counterA()` had already been called twice?

*Answer: Each call to `makeCounter()` creates a brand NEW, independent `count` variable via its own closure — `counterA` and `counterB` each close over their own separate count, so they don't share state at all.*

2. Why is event delegation especially well-suited to an app like Tiffin that constantly replaces big chunks of the DOM via `innerHTML`?

*Answer: A listener attached directly to an individual button gets destroyed the moment that button's HTML is replaced, and would need to be manually re-attached after every re-render. A single delegated listener on `document` is never attached to the elements being replaced, so it keeps working automatically for any new HTML inserted later, with zero extra setup.*

3. What does `el.closest('[data-action]')` return if the clicked element has no such ancestor at all, not even itself?

*Answer: `null` — which is exactly why the very first line inside the handler is `if (!el) return;`, to safely ignore clicks that didn't land on anything actionable.*

## 27. JavaScript — Error Handling

### 27.1 try / catch / finally

```
try {
  const data = JSON.parse(someText); // this line MIGHT throw, if someText isn't valid JSON
  console.log(data);
} catch (err) {
  console.log('Failed to parse:', err.message); // runs ONLY if something in try{} threw
} finally {
  console.log('This always runs, error or not.');// optional -- cleanup code goes here
}
```

Code that might fail goes inside `try`. If any line inside it throws an error, execution immediately jumps to `catch` — none of the remaining lines in `try` run. Without a surrounding `try/catch`, a thrown error would crash the whole script (specifically, unwind up the call stack until nothing catches it, then get logged as an uncaught error and stop that entire chain of execution).

### 27.2 throw — raising your own errors

```
function requireName(name) {
  if (!name) {
    throw new Error('Name is required');
  }
  return name;
}
```

### 27.3 Where this app actually uses try/catch, and why

Tiffin wraps exactly the operations that can genuinely fail for reasons outside its control: `JSON.parse()` on a user-supplied backup file (which might not even be valid JSON at all — Chapter 24 & 35 in Part 2), and `localStorage.setItem()` (which can throw if storage is disabled or full, Chapter 24). In both cases, the `catch` block sets a flag or shows a friendly toast message instead of letting the whole app crash over one bad file or one storage quirk.

```
function importFromFile(file) {
  const reader = new FileReader();
  reader.onload = () => {
    try {
      const parsed = JSON.parse(reader.result);
      // ...apply it...
    } catch (e) {
      toast("That file couldn't be read as a Tiffin backup."); // graceful failure, not a crash
    }
  };
  reader.readAsText(file);
}
```

#### Check your understanding — Error Handling

1. If a `try` block's second line throws an error, does the `try` block's third line still run?

*Answer: No — the moment an error is thrown, execution immediately jumps out of the try block and into the matching catch block; any remaining lines in try are skipped entirely.*

2. What happens if a thrown error is never caught by any try/catch?

*Answer: It propagates ('unwinds') up through every calling function until it either finds a catch block somewhere, or reaches the top with none — at which point it becomes an uncaught error, gets logged to the console, and execution of that particular call chain stops.*

3. Why does Tiffin wrap JSON.parse() in a try/catch when importing a backup file, specifically?

*Answer: Because the file's content is entirely outside the app's control — a user could select any file, including one that isn't valid JSON at all, which would make JSON.parse throw. Catching that lets the app show a friendly error message instead of crashing the whole page over one bad file.*

## 28. JavaScript — Async JavaScript

### 28.1 The problem: some things take time

Asking for camera access, reading a file, waiting for a network response — these can't complete instantly. If JavaScript simply paused and waited ("blocked") for each one, the entire page would freeze — no scrolling, no clicking, nothing — until it finished. Browsers avoid this with **asynchronous** (async) programming: code that starts a slow operation, then keeps the page responsive while it's in progress, and runs some follow-up code once it eventually finishes.

### 28.2 Callbacks — the original approach

```
setTimeout(function() {  
  console.log('3 seconds later...');  
}, 3000);  
console.log('This prints FIRST, immediately -- setTimeout does not block.');
```

A callback is simply a function handed to another function, to be called later once something finishes. This works, but nesting many async steps this way ("do this, and when it's done, do that, and when THAT's done...") produces deeply nested, hard-to-read code nicknamed "callback hell" — which is exactly the problem Promises were introduced to solve.

### 28.3 Promises

A **Promise** is an object representing a value that isn't ready yet, but will be at some point (or will fail). It has two possible outcomes you can attach handlers for: `.then()` (runs on success, with the resulting value) and `.catch()` (runs on failure, with the error).

```
scanDetector.detect(video)  
  .then(codes => { if (codes.length) handleBarcode(codes[0].rawValue); })  
  .catch(() => { /* detection failed this frame -- just try again next frame */ });
```

This exact `.then()/catch()` pattern appears in Tiffin's barcode-scanning loop (Chapter 34 in Part 2), where `BarcodeDetector.detect()` returns a Promise each time it examines one video frame.

### 28.4 async/await — Promises with synchronous-looking syntax

```
async function startCamera() {  
  try {  
    const stream = await navigator.mediaDevices.getUserMedia({ video: true });  
    video.srcObject = stream;  
  } catch (err) {  
    showMessage("Camera access isn't available here.");  
  }  
}
```

Marking a function `async` lets you use `await` inside it. `await somePromise` PAUSES that function at that exact line — without freezing the page, other code keeps running — and resumes it with the resolved value once the Promise finishes. This is really just different-looking syntax for the exact same `.then()` mechanism underneath; it's popular because it reads top-to-bottom like ordinary, synchronous code, and — critically —

lets you use an ordinary try/catch (Chapter 27) around it for error handling, instead of a separate `.catch()` chain.

## 28.5 requestAnimationFrame — a special-purpose async scheduling tool

```
function scanLoop() {
  if (!scanning) return;
  scanDetector.detect(video).then(codes => {
    if (codes.length) { handleBarcode(codes[0].rawValue); }
    else { requestAnimationFrame(scanLoop); } // check again right before the next repaint
  });
}
```

`requestAnimationFrame(fn)` schedules `fn` to run once, right before the browser's next screen repaint (typically ~60 times per second). Calling it again from inside itself, as above, creates a self-sustaining loop that automatically paces itself to the screen's refresh rate, and — as a bonus — automatically pauses when the browser tab isn't visible, unlike a fixed `setInterval` timer would.

### Check your understanding — Async JavaScript

1. Why doesn't `await somePromise` freeze the whole page while it's waiting?

*Answer: await only pauses the execution of the SPECIFIC async function it's inside — the rest of the page (other event handlers, rendering, user interaction) keeps working normally in the meantime. It's cooperative pausing of one function, not a full-page freeze.*

2. What are the two things you can attach to a Promise to handle its eventual outcome?

*Answer: .then(onSuccess) for when it resolves successfully, and .catch(onError) for when it fails/rejects.*

3. Is `async/await` a completely different mechanism from Promises, or built on top of them?

*Answer: Built on top of them — async/await is syntax that makes working with Promises read more like ordinary synchronous code (and lets you use try/catch for errors), but under the hood it's the exact same Promise mechanism.*

## 29. JSON — The Data Format, In Depth

### 29.1 JSON syntax rules, precisely

JSON (JavaScript Object Notation) looks almost exactly like JavaScript object/array literals, but with stricter rules — these are exactly the mistakes that make a JSON file invalid:

- Every key **MUST** be wrapped in double quotes — `{name: "Rice"}` is invalid JSON (even though it's valid JS); it must be `{"name": "Rice"}`.
- Strings must use double quotes, never single quotes.
- No comments are allowed anywhere — not `//`, not `/* */`.
- No trailing commas — `[1, 2, 3, ]` is invalid; the comma after the last item must be removed.
- Only these value types are allowed: string, number, boolean, null, array, or another (nested) object — no functions, no undefined, no dates as a real type (dates get stored as plain strings by convention).

```
{
  "id": "rice-cooked-white",
  "name": "Rice, cooked (Chawal, white)",
  "kcal": 200,
  "isVegetarian": true,
  "allergens": null,
  "tags": ["staple", "gluten-free"],
  "nested": { "servingGrams": 150 }
}
```

### 29.2 Converting between JSON text and live JavaScript values

```
const data = { name: "Alice", age: 30 };

const text = JSON.stringify(data);           // '{"name":"Alice","age":30}' -- now a STRING
const text2 = JSON.stringify(data, null, 2); // same, but pretty-printed with 2-space indentation

const back = JSON.parse(text);               // { name: "Alice", age: 30 } -- a live object again
JSON.parse("not valid json{{{");            // throws a SyntaxError -- always wrap in try/catch
                                           // when parsing anything not written by your own code
```

### 29.3 The three separate jobs JSON does in this project

- **food\_db.json** — a static, 332-item database shipped with the app (Chapter 22 in Part 2).
- **localStorage persistence** — the user's live STATE object is `JSON.stringify`'d to save, and `JSON.parse`'d to load (Chapter 24 in Part 2) — this is JSON acting as a serialization format for an in-memory JavaScript value, not a file someone hand-writes.
- **Configuration files** — `package.json` and `capacitor.config.json` are JSON files that tools (npm, Capacitor) read to configure themselves; you edit these directly as text.

#### Check your understanding — JSON

1. Is `{name: 'Rice'}` (single quotes, unquoted key) valid JSON?

*Answer: No — JSON requires double quotes for both keys and string values, with no exceptions. That exact object would need to be written {"name": "Rice"} to be valid JSON (though it IS valid as a plain JavaScript object literal, which has looser rules).*

2. What does JSON.stringify(obj, null, 2) do differently from JSON.stringify(obj)?

*Answer: The third argument (2) tells it to pretty-print the output with 2-space indentation, producing human-readable, multi-line JSON text, rather than one dense, unindented line.*

3. Why should JSON.parse() almost always be wrapped in a try/catch when the input isn't something your own code just generated?

*Answer: Because JSON.parse throws a SyntaxError if given text that isn't valid JSON — and any input coming from outside your own program (a user-selected file, pasted text, etc.) can't be trusted to always be well-formed.*

## 30. Browser Storage — localStorage, sessionStorage & Cookies

### 30.1 localStorage, revisited

```
localStorage.setItem('key', 'value'); // save (value MUST be a string)
localStorage.getItem('key');          // load, or null if never set
localStorage.removeItem('key');        // delete one key
localStorage.clear();                  // delete everything for this site
```

### 30.2 How localStorage compares to its alternatives

|                                   | localStorage | sessionStorage | Cookies             |
|-----------------------------------|--------------|----------------|---------------------|
| Persists after tab closes?        | Yes          | No             | Yes (until expiry)  |
| Shared across tabs?               | Yes          | No (per-tab)   | Yes                 |
| Typical size limit                | ~5-10 MB     | ~5-10 MB       | ~4 KB               |
| Sent to the server automatically? | Never        | Never          | Yes, every request! |

localStorage is the right choice for Tiffin specifically because it persists indefinitely and never gets automatically transmitted anywhere — cookies, by contrast, get attached to every single request a browser sends to a matching server, which is exactly why they're the traditional mechanism for server-side logins, but would be pure unnecessary overhead (and a very cramped 4KB limit) for an app with no server to send anything to in the first place.

### 30.3 Same-origin: storage is scoped per website

localStorage is isolated per **origin** (protocol + domain + port) — a page on `example.com` can never read localStorage written by `other-site.com`. This is a fundamental browser security boundary, not something Tiffin has to implement itself.

#### Check your understanding — Browser Storage

1. Your data needs to survive the user closing and reopening their browser tomorrow. localStorage or sessionStorage?

*Answer: localStorage — sessionStorage is cleared the moment its tab closes, while localStorage persists indefinitely until explicitly cleared.*

2. Why would cookies be a poor fit for storing Tiffin's ~40-field settings object, even ignoring size limits?

*Answer: Cookies are automatically sent to the server with EVERY matching request — but Tiffin has no server to send anything to at all, so that behavior would be pure wasted overhead with zero benefit, on top of cookies' much smaller (~4KB) typical size limit.*

## 31. Browser & Web APIs

### 31.1 What a 'Web API' is

Beyond the core DOM (Chapter 24), browsers expose dozens of additional built-in capabilities — collectively called Web APIs — that JavaScript can call into. None of these require any external library; they're simply available as global objects/functions the moment your script runs in a browser.

### 31.2 BarcodeDetector

Scans a video frame (or image) for barcodes and reports what it finds, entirely built into supporting browsers — no external barcode-scanning library needed. Not universally supported, so real code always feature-detects first:

```
if ('BarcodeDetector' in window) {  
  // safe to use it  
} else {  
  // fall back to manual entry  
}
```

### 31.3 MediaDevices.getUserMedia

Requests permission to access the camera and/or microphone, returning a live "MediaStream" that can be piped into a <video> element for display. This ALWAYS requires explicit user permission — the browser shows its own native "Allow camera access?" prompt, which JavaScript cannot bypass or auto-approve.

### 31.4 FileReader

Reads the contents of a local file the user has selected (via a file-input element), asynchronously via an on load callback, so reading a large file doesn't freeze the page.

```
const reader = new FileReader();  
reader.onload = () => {  
  const text = reader.result;    // the file's full text content, once loading finishes  
};  
reader.readAsText(file);        // starts the (async) read
```

### 31.5 Clipboard API

```
navigator.clipboard.writeText('some text')  
  .then(() => console.log('copied!'))  
  .catch(() => console.log('clipboard access was denied'));
```

### 31.6 Chart.js — a third-party library, not a Web API

Worth distinguishing clearly: Chart.js is NOT built into the browser — it's an external, open-source JavaScript library that has to be downloaded (Tiffin fetches it from a CDN, Chapter 28 in Part 2) before its Chart object becomes available to use, unlike everything else in this chapter, which is simply always present.

## Check your understanding — Browser & Web APIs

1. Why does code always check `if ('BarcodeDetector' in window)` before using it, rather than just calling it directly?

*Answer: Because it's not supported in every browser — feature-detecting first lets the app gracefully fall back to a manual-entry alternative instead of crashing with an error in browsers that lack support.*

2. Can JavaScript silently grant itself camera access without the user seeing a permission prompt?

*Answer: No — `getUserMedia` always requires the browser's own native permission prompt, which the user must explicitly approve; there is no way for a website's JavaScript to bypass or auto-approve this.*

3. Is Chart.js a browser built-in Web API, or a separate library that has to be loaded?

*Answer: A separate, third-party library — it has to be fetched (e.g. from a CDN) and loaded as a script before its functionality becomes available, unlike true Web APIs like `FileReader` or `BarcodeDetector`, which are simply always present in a supporting browser.*

## 32. Python — From Zero

### 32.1 Why a second language at all?

Recall from the project tour (Part 2 will cover this fully): this codebase uses Python for exactly one small job — a build script that glues the app's source files together. Python and JavaScript are different languages with different syntax, but the underlying programming CONCEPTS (variables, functions, loops, conditionals) are the same ideas you already learned in Chapters 14-19 — only the punctuation changes.

### 32.2 Indentation instead of curly braces

This is Python's single most distinctive feature: instead of `{ }` marking a block of code, Python uses consistent INDENTATION (spaces) itself to mark where a block starts and ends. Getting indentation wrong is a real syntax error in Python, not just a style nitpick.

```
# JavaScript:
if (age >= 18) {
  console.log("adult");
} else {
  console.log("minor");
}

# The same logic in Python:
if age >= 18:
    print("adult")
else:
    print("minor")
# Note: no curly braces, no semicolons -- the colon (:) starts a block,
# and everything indented beneath it belongs to that block.
```

### 32.3 Variables and types

```
name = "Alice"          # no `let`/`const` -- Python variables are just assigned directly
age = 30                 # int
height = 5.6            # float
is_active = True        # bool -- capitalized True/False, unlike JS's lowercase true/false
nothing = None          # Python's equivalent of JS's null
```

### 32.4 Lists and dictionaries — Python's arrays and objects

```
fruits = ["apple", "banana", "cherry"]  # a LIST -- like a JS array
fruits[0]                                # "apple"
fruits.append("date")                    # like JS's .push()
len(fruits)                              # like JS's .length, but it's a FUNCTION, not a property

person = {"name": "Alice", "age": 30}    # a DICT -- like a JS object
person["name"]                           # "Alice" -- Python ALWAYS uses bracket notation,
                                           # there's no dot-notation shortcut like JS has
person.get("age")                         # 30 -- safer than [] if the key might not exist
```

### 32.5 Functions

```
def add(a, b):          # `def` instead of `function`
    return a + b

def greet(name="friend"): # default parameter value, same idea as JS
    return f"Hello, {name}" # an f-string -- Python's equivalent of a JS template literal
```

`f"Hello, {name}"` is Python's direct equivalent of JavaScript's ``Hello, ${name}`` — the `f` prefix before the opening quote is what tells Python to evaluate the `{...}` expressions inside the string.

## 32.6 Loops

```
for fruit in fruits:          # like JS's for...of
    print(fruit)

for i in range(5):            # like a classic for(let i=0; i<5; i++) loop
    print(i)                  # prints 0, 1, 2, 3, 4

count = 0
while count < 3:              # works exactly like JS's while
    print(count)
    count += 1                # Python HAS +=, but no ++ shorthand -- must write count += 1
```

## 32.7 Importing code from other modules

```
import pathlib              # brings in Python's built-in "pathlib" module, used as pathlib.Path(...)
import hashlib              # used for computing hashes (like SHA-256)
import base64              # used for base64 encoding/decoding
import argparse            # used for parsing command-line arguments

from pathlib import Path    # imports just ONE specific name directly, so you can write Path(...)
                             # instead of pathlib.Path(...)
```

This is conceptually the same idea as JavaScript's `<script src="...">` or, in more modern JS, `import` statements — pulling in code someone else already wrote so you don't have to reimplement it. Python ships with a huge standard library of these built-in modules; `pathlib`, `hashlib`, `base64`, and `argparse` are all part of it — `build.py` needs no separate installation step for any of them.

## 32.8 Reading and writing files

```
from pathlib import Path

text = Path("app.js").read_text(encoding="utf-8") # read an entire file as one string
Path("output.txt").write_text("hello", encoding="utf-8") # overwrite a file with new content
```

`encoding="utf-8"` specifies exactly how the file's bytes should be interpreted as text — UTF-8 is the standard, universal text encoding capable of representing every character in every language, and specifying it explicitly (rather than relying on a possibly inconsistent system default) is considered best practice for reliable, portable code.

### Check your understanding — Python Basics

1. What defines a block of code in Python, if not curly braces?

*Answer: Consistent indentation (whitespace) — everything indented at the same level beneath a line ending in a colon (like ``if x:`` or ``def f():``) belongs to that block. Inconsistent indentation is a real syntax error.*

2. What's the Python equivalent of a JavaScript object literal, `{name: 'Alice'}`?

*Answer: A dict (dictionary): `{"name": "Alice"}` — read with `person["name"]` (Python always uses bracket notation for dicts; there's no dot-notation shortcut the way JS objects have).*

3. What does the `f` before a string like `f"Hello, {name}"` do?

*Answer: It marks the string as an f-string, telling Python to evaluate any `{expression}` inside it and substitute the result — the direct Python equivalent of a JavaScript template literal's `${...}`.*

## 33. Python Applied — Reading build.py With Full Understanding

With Chapter 32's vocabulary in hand, build.py — covered in full architectural detail in Chapter 19 of Part 2 — is now fully readable. A quick preview of its shape:

```
import argparse, base64, hashlib, pathlib

ROOT = pathlib.Path(__file__).parent    # the folder this script itself lives in

def sha256_b64(text):
    # a function, just like JS -- `def` instead of `function`
    return base64.b64encode(hashlib.sha256(text.encode("utf-8")).digest()).decode("ascii")

def build(output_path):
    shell = (ROOT / "shell.html").read_text(encoding="utf-8")    # read a file's text (Ch.32.8)
    css = (ROOT / "app.css").read_text(encoding="utf-8")
    ...
    out = (
        shell.replace("__CSP__", csp)            # find-and-replace, like JS's .replace() with a
        .replace("/__CSS__/", css)              # PLAIN STRING (not a regex) as the search target
        .replace("/__FOODDB__/", fooddb)
        .replace("/__APPJS__/", appjs)
    )
    output_path = pathlib.Path(output_path)
    output_path.write_text(out, encoding="utf-8")    # write the assembled file out (Ch.32.8)

if __name__ == "__main__":
    # only runs when this file is executed directly
    parser = argparse.ArgumentParser()           # (not when imported as a module elsewhere)
    parser.add_argument("-o", "--output", default="dist/tiffin.html")
    args = parser.parse_args()
    build(args.output)
```

`ROOT / "shell.html"` is Python's `pathlib` module cleverly overloading the `/` operator to mean "join a path segment" — `ROOT / "shell.html"` builds a full file path regardless of whether the underlying operating system uses forward or backward slashes, which is exactly the kind of small cross-platform detail a purpose-built module like `pathlib` handles for you.

`if __name__ == "__main__":` is a very common Python idiom worth recognizing on sight: it's Python's way of saying "only run the code below if this specific file was the one directly executed (e.g. via `python3 build.py`), not if some OTHER script merely imported functions from this file." It has no direct JavaScript equivalent, since browser scripts don't have quite the same "run directly vs. imported" distinction.

### Check your understanding — build.py

1. What does `ROOT / "shell.html"` actually do?

*Answer: It joins the ROOT path with the filename "shell.html" to build a full file path, using pathlib's overloaded / operator — a cross-platform-safe way to combine path segments without manually worrying about forward vs. backward slashes.*

2. What is `.replace("__CSP__", csp)` doing, in terms you already know from JavaScript?

*Answer: Exactly the same thing as JavaScript's string .replace() method with a plain string as the search argument: find the first occurrence of the literal text "\_\_CSP\_\_" and substitute it with the value of the `csp` variable.*

## 34. Git — Version Control Basics

### 34.1 The problem Git solves

Without version control, tracking changes to a project means either manually copying files ("app\_v2\_final\_FINAL.js") or simply losing track of what changed and why. Git solves this by recording a complete, navigable HISTORY of every change ever made to a project.

### 34.2 The core concepts

- **Repository ("repo")** — a project folder Git is tracking the history of.
- **Commit** — a saved snapshot of the entire project at one point in time, with a message describing what changed and why.
- **Branch** — an independent line of development; you can make commits on a branch without affecting the project's main, stable line of history until you're ready to combine them.
- **Remote** — a copy of the repository hosted elsewhere (commonly on GitHub), that your local copy can sync with.

### 34.3 The everyday command sequence

```
git status          # what's changed since the last commit?
git add app.js      # stage a specific file's changes, marking them "ready to commit"
git commit -m "Fix streak calculation off-by-one" # save a snapshot of everything staged
git push origin main # upload local commits to the remote (here, a branch named "main")
git pull origin main # download and merge new commits FROM the remote
git log             # view commit history
git diff            # see exactly what changed, line by line, not yet committed
```

### 34.4 Branching and merging

```
git branch          # list local branches
git checkout -b new-feature # create AND switch to a new branch in one step
git checkout main    # switch back to the main branch
git merge new-feature # bring new-feature's commits into whatever branch you're on now
```

Branches let multiple lines of work — a bug fix, a new feature, an experiment — proceed independently without interfering with each other, then be combined (merged) once ready.

### 34.5 Git vs. GitHub — a common point of confusion

**Git** is the version-control TOOL itself — it works entirely on your own computer and needs no internet connection or external account. **GitHub** is a separate, unrelated COMPANY'S hosting SERVICE for Git repositories — it stores a remote copy of your repo, provides a web interface for browsing it, and adds collaboration features on top (like pull requests, covered next, and GitHub Actions, Chapter 35) that have nothing to do with Git itself. You could use Git your entire life and never touch GitHub; other hosting services (GitLab, Bitbucket) exist too.

## 34.6 Pull requests

A **pull request** (PR) is a GitHub feature (not a Git concept) proposing that the commits on one branch be merged into another — typically used to have someone else review a change before it joins the main line of the project. This project's own workflow (referenced throughout the setup that generated the automated Android/iOS builds) is exactly this: work happens on a feature branch, and a PR proposes merging it into main.

### Check your understanding — Git

1. What's the difference between ``git add`` and ``git commit``?

*Answer: `git add` stages specific changes, marking them as ready to be included in the next commit. `git commit` actually saves a permanent snapshot of everything currently staged, along with a message describing the change.*

2. Is GitHub a part of Git itself, or a separate product?

*Answer: A separate, unrelated hosting service built on top of Git — Git works entirely locally with no internet or account needed; GitHub is one of several companies that host remote copies of Git repositories and add extra features (pull requests, Actions, issue tracking) on top.*

3. Why use branches instead of just making every commit directly on the main line of history?

*Answer: Branches let independent lines of work (a feature, a fix, an experiment) proceed without interfering with the stable main branch, and let that work be reviewed (e.g. via a pull request) before it's merged in.*

## 35. GitHub Actions — CI/CD Basics

### 35.1 What CI/CD means

**Continuous Integration** (CI) means automatically building and testing code every time it changes, catching problems immediately rather than discovering them later. **Continuous Deployment/Delivery** (CD) extends this to automatically packaging and shipping the result. GitHub Actions is GitHub's own built-in tool for both.

### 35.2 YAML — the configuration format these workflows are written in

YAML is a plain-text format for structured configuration, using indentation for nesting (similar in spirit to Python, Chapter 32.2) and a simple `key: value` syntax. A dash (-) at the start of a line marks an item in a list.

```
name: Build Android App           # a key: value pair

on:                               # `on` is a key whose value is a nested object
  push:                           # a key whose value is a LIST, written inline with [ ]
    branches: [ main, my-branch ]

jobs:                             # `jobs` is a key whose value is a nested object of named jobs
  build:                          # this job runs on a fresh Ubuntu Linux virtual machine
    runs-on: ubuntu-latest        # a LIST of steps, each starting with `-`
    steps:                        # step 1: use someone else's prebuilt "action"
      - uses: actions/checkout@v4 # step 2: run a raw shell command
      - run: npm install
      - run: npm run build
```

### 35.3 The core vocabulary

- **Workflow** — one complete YAML file describing an automated process (this project has two: one for Android, one for iOS).
- **Trigger** (the `on:` key) — what causes the workflow to run; most commonly `push` (code was pushed) or `pull_request` (a PR was opened/updated).
- **Job** — a set of steps that all run together on one freshly-provisioned virtual machine (the **runner**).
- **Step** — one action within a job: either `run:` (an arbitrary shell command) or `uses:` (a reusable, prebuilt "action" someone else published, referenced like `actions/checkout@v4`).
- **Artifact** — a file (or folder) produced by a workflow run that gets saved and made downloadable afterward — this is exactly how the compiled `.apk` and `.ipa` files become available once a build finishes.

### 35.4 Why this matters for a project with no dedicated build server

Without CI/CD, producing a signed Android build would require someone to manually install Android Studio, the right SDK versions, and run the build locally, every single time — slow, error-prone, and impossible to do from, say, a phone. GitHub Actions instead spins up a brand-new, identically-configured virtual machine for every single run, runs the exact same recipe every time, and throws the machine away afterward — this determinism is a large part of CI/CD's value: it eliminates "works on my machine" inconsistency entirely.

[Check your understanding — GitHub Actions](#)

1. In YAML, how do you represent a list of items, like several branch names?

*Answer: Either inline with square brackets, e.g. branches: [ main, dev ], or as multiple lines each starting with a dash (-), indented under the key.*

2. What's the difference between a workflow's `run:` step and its `uses:` step?

*Answer: run: executes an arbitrary raw shell command you write yourself. uses: invokes a reusable, prebuilt 'action' that someone else already published (like actions/checkout, which handles downloading the repo's code) — you don't need to write that logic yourself.*

3. Why does a fresh virtual machine get created for every single workflow run, rather than reusing one machine repeatedly?

*Answer: To guarantee a clean, identical, reproducible environment every time, eliminating 'works on my machine' inconsistencies where leftover state from a previous run might cause a build to succeed or fail unpredictably.*

## 36. Capacitor — Turning a Website into a Phone App

### 36.1 Native vs. web vs. hybrid apps

A **native** app is written directly in a platform's own language and frameworks (Swift/Kotlin, typically) and can access every device feature directly, but must be written and maintained twice — once per platform. A **web** app is just a website, worked with entirely through the vocabulary of this whole guide, but can't be installed from an app store or (traditionally) access certain device features. A **hybrid** app is a middle path: write it once as a web app, then wrap that web app inside a thin native shell for each platform, getting app-store installability with a single shared codebase.

### 36.2 What a WebView actually is

A WebView is a native, embeddable browser component that any native app can include as part of its own interface — essentially a full-screen browser tab with no address bar or browser chrome visible around it. Capacitor generates a minimal native app, for each platform, whose entire screen is just one WebView pointed at your built web app's HTML file. Your JavaScript runs inside that WebView exactly as it would in a normal browser tab — the same DOM, the same events, the same `localStorage`.

### 36.3 The Capacitor workflow, concretely

```
// capacitor.config.json
{
  "appId": "com.example.tiffin",
  "appName": "Tiffin",
  "webDir": "dist"      // <-- the folder containing the already-built web app
```

- 1. `npm run build` assembles the single `dist/index.html` file (Chapter 19).
- 2. `npx cap sync android` (or `ios`) copies that file into a native project template Capacitor generated once, ahead of time.
- 3. The platform's own native compiler — Gradle for Android, Xcode for iOS — builds an actual, installable native app from that template, with the WebView pointed at your file.

### 36.4 Why this required almost no extra code for Tiffin specifically

Because Tiffin already ran entirely client-side with zero server dependency (Chapter 38 of Part 2 covers this decision in full), wrapping it in a WebView changed essentially nothing about how the app behaves — the exact same `localStorage`-based persistence, the exact same JavaScript logic, works completely unmodified inside the native shell. An app that DID depend on a server would need to keep working through the WebView's networking exactly as it does in a normal browser too, which is usually fine, but would be one more thing to verify.

#### Check your understanding — Capacitor

1. What is a WebView, concretely?

*Answer: A native, embeddable browser component — essentially a full-screen browser tab with no visible address bar or browser UI around it — that a native app can include as part of its own interface.*

2. Why did wrapping Tiffin with Capacitor require almost no code changes, specifically?

*Answer: Because Tiffin already had zero server dependency and stored everything in localStorage — exactly the same behavior that already worked in a normal desktop browser tab continues to work completely unmodified inside a WebView, since a WebView provides the same DOM, JavaScript engine, and localStorage that any browser tab does.*

3. Name the three broad categories of mobile app (native / web / hybrid) and which one Capacitor produces.

*Answer: Native (platform-specific code, e.g. Swift/Kotlin), web (a website, not installable from an app store), and hybrid (a web app wrapped in a thin native shell) — Capacitor produces hybrid apps.*

# PART 2

## The App, Explained End to End

With that vocabulary in hand, we can now read every file in this project and understand exactly what it does and why. We'll go file by file, then work through `app.js` — the largest and most important file — section by section, in the order it's actually written.

## 37. Project Tour — Every File and What It's For

| File                     | Language   | Job                                                           |
|--------------------------|------------|---------------------------------------------------------------|
| shell.html               | HTML       | The empty page skeleton — a few containers JS fills in        |
| app.css                  | CSS        | All visual styling, including dark mode and responsive layout |
| app.js                   | JavaScript | The entire application logic — ~1,700 lines, one big IIFE     |
| food_db.json             | JSON       | 332 pre-loaded foods with full nutrition data                 |
| build.py                 | Python     | Glues the four files above into one dist/index.html           |
| package.json             | JSON       | Project metadata + npm scripts + dependency list              |
| capacitor.config.json    | JSON       | Tells Capacitor which folder is the built web app             |
| .github/workflows/*.yaml | YAML       | CI/CD recipes that auto-build Android/iOS apps                |

Notice what's *not* here: no backend server code, no database server, no API. Everything the app needs — the food database included — ships as static files, and all logic runs inside the user's own browser. This single decision shapes nearly everything else about the project (Chapter 57).

## 38. The Build Pipeline — build.py Deep Dive

shell.html contains four placeholder markers that aren't real HTML — they're just text build.py knows to search and replace:

```
<style>
/*__CSS__*/
</style>
...
<script>
const FOOD_DB = /*__FOODDB__*/;
</script>
<script>
/*__APPJS__*/
</script>
```

build.py reads all four source files, then does a simple find-and-replace for each placeholder, producing one giant, fully self-contained HTML file at dist/index.html — CSS, food data, and app logic all inlined directly into the page, no separate file requests needed:

```
shell = (ROOT / "shell.html").read_text(encoding="utf-8")
css = (ROOT / "app.css").read_text(encoding="utf-8")
appjs = (ROOT / "app.js").read_text(encoding="utf-8")
fooddb = (ROOT / "food_db.json").read_text(encoding="utf-8")

out = (
    shell.replace("__CSP__", csp)
    .replace("/*__CSS__*/", css)
    .replace("/*__FOODDB__*/", fooddb)
    .replace("/*__APPJS__*/", appjs)
)
output_path.write_text(out, encoding="utf-8")
```

Why bother splitting the source into 4 files just to glue them back together? **Developer ergonomics.** Editing app.js in a dedicated .js file gets you syntax highlighting, linting, and autocomplete in a code editor — none of which work well for JS embedded inside an HTML string. Splitting the source and reassembling it at build time gets the best of both: a nice editing experience, and a single zero-dependency HTML file to actually ship.

### 38.1 The Content-Security-Policy (CSP) — a security feature

The trickiest part of build.py computes a cryptographic **SHA-256 hash** (a unique fingerprint) of the exact text inside each of the two inline <script> tags:

```
def sha256_b64(text):
    return base64.b64encode(hashlib.sha256(text.encode("utf-8")).digest()).decode("ascii")

script_hashes = " ".join(
    f"sha256-{sha256_b64(body)}" for body in (fooddb_script_body, appjs_script_body)
)
```

This hash gets embedded into a Content-Security-Policy HTTP header/meta tag that tells the browser: "only run inline scripts whose exact content matches one of these fingerprints — refuse to run anything else." If an attacker somehow injected a rogue <script> tag into the page (Chapter 58 explains how that could even happen), its hash wouldn't match, and the browser would simply refuse to execute it. This is a strong,

defense-in-depth security measure explained fully in Chapter 58.

## 39. shell.html Deep Dive

Once you strip out the placeholders, shell.html is almost nothing — deliberately so:

```
<div class="topbar">...</div>                <!-- mobile-only top bar -->
<div class="app">
  <nav class="sidenav" id="sideNav"></nav>      <!-- EMPTY -- JS fills this -->
  <main class="main">
    <div id="viewRoot"></div>                  <!-- EMPTY -- JS fills this -->
  </main>
</div>
<nav class="bottomnav" id="bottomNav"></nav>    <!-- EMPTY -- JS fills this -->
<div class="modal-overlay" id="modalOverlay">
  <div class="modal" id="modalContent"></div>    <!-- EMPTY -- JS fills this -->
</div>
<div class="toast-wrap" id="toastWrap"></div>    <!-- EMPTY -- JS fills this -->
```

Five containers, each with an `id`, and every single one starts completely empty. This is the hallmark of a JavaScript-driven single page app: HTML provides only the *slots*; JavaScript generates 100% of what actually appears inside them, and keeps regenerating it as the app's data changes. We'll see exactly how in Chapter 44.

## 40. app.css Deep Dive

### 40.1 Design tokens and dark mode, revisited

Recall from Part 1 that CSS variables let you name a value once and reuse it everywhere. app.css defines its full palette under `:root`, then *redefines the exact same variable names* inside a dark-mode block:

```
:root {
  --paper: #FBF3E7; /* warm cream background */
  --text: #241C12; /* near-black text */
  --accent: #FF6B4A; /* the brand orange */
}

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --paper: #161210; /* near-black background */
    --text: #F5ECDF; /* warm off-white text */
    --accent: #FF7D5E; /* a slightly brighter orange, for contrast on dark */
  }
}

:root[data-theme="dark"] {
  /* the SAME dark values, but applied whenever the app's own theme setting
   is explicitly "dark" -- regardless of the OS setting */
  --paper: #161210; --text: #F5ECDF; --accent: #FF7D5E;
}
```

Three theme states are supported: **system** (follow the OS's light/dark preference automatically, via the `@media (prefers-color-scheme: dark)` query), or an explicit **light** / **dark** override, applied by JavaScript setting a `data-theme` attribute on the `<html>` element (Chapter 54 shows the JS side of this). Not one other CSS rule in the whole file needs to know or care which theme is active — they all just say `color: var(--text)`, and the variable's current value does the rest.

### 40.2 Mobile vs desktop layout

Recall the media query pattern from Part 1. app.css uses exactly one breakpoint, 860px, to switch between a desktop sidebar and a mobile bottom tab bar:

```
.sidenav { width: 216px; /* ...desktop sidebar styles... */ }
.bottomnav { display: none; /* hidden by default (desktop) */ }

@media (max-width: 860px) {
  .sidenav { display: none; } /* hide sidebar on narrow screens */
  .bottomnav { display: flex; } /* show the tab bar instead */
  .topbar { display: flex; } /* show the mobile top bar */
}
```

Both navigation elements exist in the HTML at all times — CSS alone decides which one is visible, based purely on the browser window's width, no JavaScript involved.

### 40.3 Reduced motion accessibility

```
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after {  
    animation-duration: .001ms !important;  
    transition-duration: .001ms !important;  
  }  
}
```

Some users configure their OS to minimize animations (often for motion-sensitivity/vestibular reasons). This query respects that system preference by effectively disabling all CSS animations and transitions for them, app-wide, in one rule.

## 41. food\_db.json Deep Dive

This file is a single JSON array containing 332 food objects. Every object has the exact same set of keys — the same 44 nutrient fields, always present (even if zero), plus a few descriptive fields:

```
{
  "id": "rice-cooked-white",
  "name": "Rice, cooked (Chawal, white)",
  "category": "Indian",
  "servingLabel": "1 cup",
  "servingGrams": 150,
  "kcal": 200,
  "protein": 4.2,
  "carbs": 44,
  "fat": 0.4,
  "satFat": 0.1,
  "fiber": 0.6,
  ... (continues through all 44 nutrients -- vitamins, minerals, fats, etc.)
}
```

- `id` — a unique, human-readable identifier (used to reference this food from the diary, never the display name, since names could change).
- `category` — 'Indian' or 'International', used for the filter chips in the Foods tab.
- `servingLabel` / `servingGrams` — a human-friendly serving description ("1 cup") paired with its weight in grams, so any other serving size can be computed proportionally.
- Every other key matches one entry in the app's own `NUTRIENT_KEYS` list (Chapter 42) — this consistency is what lets the same generic nutrient-math functions work identically on every food.

**Note:** This file is loaded once, embedded directly as a JS array literal via `build.py`'s `__FOODDB__` placeholder (Chapter 38) — there's no network request to fetch it at runtime, which is why the app works completely offline.

## 42. app.js Part 1 — Setup & The Shape of the Data

Everything from here through Chapter 56 walks through app.js top to bottom. Line numbers refer to the actual file in this repository.

### 42.1 The wrapper (lines 1-8)

```
(function(){  
  "use strict";
```

As explained in Part 1's closures/IIFE chapter, this is an IIFE: it isolates every name the app defines from the rest of the page, and turns on JavaScript's strict mode.

### 42.2 Icons (lines 9-27)

An object literal, `ICON`, mapping short names like `home`, `flame`, `x` to raw SVG markup strings. Recall from Part 1 that `<svg>` draws vector graphics directly in markup — no image files are downloaded for any icon in this app; they're all just text, embedded directly into whatever HTML string needs them, e.g. `${ICON.flame}`.

### 42.3 Constants (lines 29-114)

```
const STORAGE_KEY = 'tiffin_state_v1'; // the localStorage key everything is saved under  
  
const NUTRIENT_KEYS = [  
  'kcal', 'protein', 'carbs', 'fat', 'satFat', 'fiber', 'sugar', 'sodium', ... // all 44, by name  
];
```

`NUTRIENT_KEYS` is the single source of truth for "what nutrients does this app track?" — every nutrient-math function (Chapter 44) loops over this list rather than hardcoding 44 separate lines, so adding a 45th nutrient in the future would mean editing this one list, not every function in the file.

```
const NUTRIENT_GROUPS = [  
  {title: 'Macros', rows: [  
    ['protein', 'Protein', 'g', 'proteinGoal'],  
    ['carbs', 'Carbohydrates', 'g', 'carbGoal'],  
    ['fat', 'Fat', 'g', 'fatGoal'],  
    ...  
  ]},  
  {title: 'Vitamins', rows: [ ... ]},  
  {title: 'Minerals', rows: [ ... ]},  
  ...  
];
```

`NUTRIENT_GROUPS` is a richer structure used purely for display: it groups related nutrients under headings (Macros, Vitamins, Minerals...) and, per nutrient, bundles its internal key, its human-readable label, its unit, and which goal setting (if any) it's measured against. This one list drives both the Nutrients tab (Chapter 48) and the custom-food-entry form (Chapter 50) — define a nutrient's metadata once, get two different screens generated from it automatically.

```

const FIXED_TARGETS = {
  addedSugarLimit: 50, omega3AlaGoal: 1.4,
  vitB1Goal: 1.2, vitB2Goal: 1.3, ... // general adult nutrition reference values
};

const ACTIVITY_MULT = {sedentary:1.2, light:1.375, moderate:1.55, active:1.725, very_active:1.9};

const DEFAULT_GOALS = {
  calorieGoal: 2000, proteinGoal: 100, carbGoal: 250, fatGoal: 65, ...
};

```

FIXED\_TARGETS holds general recommended daily amounts (for nutrients too obscure to let users customize) — merged in at display time, never stored per-user. ACTIVITY\_MULT holds the standard multipliers used in the Mifflin-St Jeor energy formula (Chapter 46). DEFAULT\_GOALS is what a brand-new user starts with before they customize anything in Profile.

## 42.4 Default-state factory functions (lines 116-133)

```

function defaultMealSlots(){
  return [
    {id:'meal-1', name:'Meal 1', emoji:'■'},
    {id:'meal-2', name:'Meal 2', emoji:'■'},
    ... // 5 meals by default
  ];
}
function defaultProfile(){
  return { name:'', sex:null, age:null, heightCm:null, ... };
}
function defaultState(){
  return {
    goals: {...DEFAULT_GOALS}, profile: defaultProfile(), diary: {}, weightLog: [],
    customFoods: [], customMeals: [], barcodeMap: {}, mealSlots: defaultMealSlots(),
  };
}

```

Notice each of these is a *function that returns a fresh object*, not a shared constant object. This matters: if `defaultState()` were instead one constant object reused everywhere, every new user (and the "Reset all data" feature, Chapter 54) would end up sharing and accidentally mutating the exact same object in memory. Calling a function guarantees a brand new, independent copy every time — recall Part 1's chapter on arrays/objects holding references, not copies.

This block also defines the complete shape of the app's data: goals (calorie/macro targets), profile (the user's body stats), diary (a dictionary of date → meals eaten), weightLog (an array of weigh-ins), customFoods / customMeals (anything the user created themselves), barcodeMap (barcode numbers linked to specific foods), and mealSlots (the user's customizable list of meal names, replacing a fixed "Breakfast/Lunch/Dinner" scheme).

## 43. app.js Part 2 — Storage & Security

### 43.1 Loading and saving (lines 150-190)

```
let STATE = loadState(); // <-- the app's ENTIRE data lives in this one variable

function saveState(){
  try{ localStorage.setItem(STORAGE_KEY, JSON.stringify(STATE)); }
  catch(e){ storageOk = false; }
}
```

This is the `localStorage` pattern from Part 1, applied to the whole app's data at once. Every single place in `app.js` that changes something — adding a food, logging weight, changing a setting — calls `saveState()` immediately afterward. There is no "Save" button for the app as a whole; changes are persisted the instant they happen.

The `try/catch` matters because `localStorage` can fail — private/incognito browsing sometimes disables it entirely, or storage could be full. Rather than crash, the app sets a flag (`storageOk = false`) and later warns the user their data won't be saved, so they know to back it up manually (Chapter 54).

### 43.2 `mergeKnownFields` — a security-conscious loader

This is the single most important function to understand for interview purposes, because it fixes a real, named security vulnerability class. Read the code and its own comment carefully:

```
// Copies only keys already present on `defaults` from `source`, so a
// stored/imported JSON blob can never introduce arbitrary keys (including
// "__proto__", which Object.assign(target, source) would otherwise use to
// repoint target's prototype -- CWE-1321 prototype pollution).
function mergeKnownFields(defaults, source){
  const out = Object.assign({}, defaults);
  if(source && typeof source === 'object'){
    Object.keys(defaults).forEach(k=>{
      if(Object.prototype.hasOwnProperty.call(source, k)) out[k] = source[k];
    });
  }
  return out;
}
```

This deserves the full explanation, because it's exactly the kind of thing an interviewer will probe. The naive way to merge "saved settings" over "default settings" would be:

```
const merged = Object.assign({}, defaults, source); // DANGEROUS with untrusted `source`
```

The problem: every JavaScript object secretly has a hidden `__proto__` property pointing to its prototype (the object it inherits shared behavior from). If `source` came from untrusted JSON — like an imported backup file someone could hand-edit, or in principle data planted by an attacker — and that JSON contained a key literally named `"__proto__"`, a naive `Object.assign` would follow it and let the attacker inject properties onto `Object.prototype` itself, corrupting the behavior of every object in the entire running app. This class of bug has an official name and ID: **CWE-1321, Prototype Pollution**.

`mergeKnownFields` defends against this in two ways at once: (1) it only ever copies keys that already exist on `defaults` — since `defaults` is always a hardcoded object this app itself wrote, `"__proto__"` is never one of its keys, so the loop simply never looks at it; and (2) it uses `Object.prototype.hasOwnProperty.call(source, k)` rather than the simpler `source.hasOwnProperty(k)` or `k in source` — the former can itself be spoofed if `source` has been tampered with to no longer inherit the normal `Object` prototype (recall Part 1's `Object` chapter); calling the trusted, original method explicitly sidesteps that.

This function is used in two places: loading state from `localStorage` on startup (which, strictly, is data the app itself wrote, so lower risk) and importing a backup file the user selects (Chapter 54) — the more realistic risk, since backup files could be shared, edited, or come from an untrusted source.

### 43.3 Legacy data migration

`loadState()` also contains a one-time migration for users who had the app before "meal slots" became customizable, converting their old fixed `breakfast/lunch/dinner/snacks` keys into the new generic `meal-1/meal-2/...` scheme automatically, so nobody's existing diary history is lost when the app is updated.

## 44. app.js Part 3 — Utility & Math Functions

### 44.1 General-purpose helpers (lines 193-210)

```
function uid(){ return Math.random().toString(36).slice(2,9); }
function round1(n){ return Math.round((n + Number.EPSILON) * 10) / 10; }
function clamp(v,a,b){ return Math.max(a, Math.min(b, v)); }
function escapeHtml(s){ return String(s).replace(/&<>"/g, c=>({...}[c])); }
```

- `uid()` generates a short random ID — used for every custom food, custom meal, and meal slot the user creates, so each can be referenced uniquely.
- `round1()` rounds to 1 decimal place, used for nearly every displayed number.
- `clamp(v, a, b)` forces a value to stay between `a` and `b` (used to keep progress bars from visually overflowing past 100%).
- `escapeHtml()` is Tiffin's central XSS defense — full explanation in Chapter 58.

### 44.2 Date helpers

```
function toKey(d){ return d.getFullYear()+'-'+pad2(d.getMonth()+1)+'-'+pad2(d.getDate()); }
function todayKey(){ return toKey(new Date()); }
function addDaysKey(key, delta){ const d = keyToDate(key); d.setDate(d.getDate()+delta); return toKey(d); }
```

The whole app represents a date not as a JavaScript Date object, but as a plain string key like `'2024-09-17'`. This makes it trivially usable as an object key (for the diary, see Chapter 45) and easy to compare/sort as plain strings, while still allowing arithmetic ("give me yesterday") by briefly converting back to a real Date object, changing it, and converting back to a key.

### 44.3 Nutrient math — the core arithmetic of the whole app

```
function zeroNutrients(){
  const o = {}; NUTRIENT_KEYS.forEach(k=>o[k]=0); return o;
}
function scaleNutrients(base, qty){
  const o = {}; NUTRIENT_KEYS.forEach(k=> o[k] = (base[k]||0) * qty); return o;
}
function addNutrients(a,b){
  const o = {}; NUTRIENT_KEYS.forEach(k=> o[k] = (a[k]||0) + (b[k]||0)); return o;
}
```

These three tiny functions are the mathematical foundation everything else builds on, and they're a perfect example of the `NUTRIENT_KEYS` list paying off: instead of 44 lines of `o.protein = a.protein + b.protein`; `o.carbs = a.carbs + b.carbs`; ..., one loop handles every current and future nutrient identically — Part 1's Arrays chapter covered exactly this `.forEach()` pattern.

- `zeroNutrients()` — an empty starting point (everything 0), used as the running total before summing a day's meals.
- `scaleNutrients(base, qty)` — multiplies every nutrient by a quantity, e.g. "1.5 servings of rice" scales rice's per-serving nutrients by 1.5.

- `addNutrients(a, b)` — adds two nutrient sets together, key by key, e.g. combining breakfast's total with lunch's total.

## 44.4 Resolving a "reference" into actual nutrition data

A logged diary entry doesn't store a copy of a food's nutrients — it stores a lightweight *reference*: `{type: 'db', id: 'rice-cooked-white'}` plus how many servings. This keeps the diary small and means editing a custom food's nutrition later automatically updates every past log entry that referenced it.

```
function resolveRefBase(ref){
  if(!ref) return null;
  if(ref.type === 'db'){
    const f = dbFoodById(ref.id); if(!f) return null;
    return {name:f.name, servingLabel:f.servingLabel, servingGrams:f.servingGrams, nutrients:f, ...};
  }
  if(ref.type === 'custom'){ ... }
  if(ref.type === 'meal'){
    // a custom "meal" (recipe) is a LIST of other refs -- resolve each ingredient
    // recursively and sum them together
    let n = zeroNutrients(); let grams = 0;
    m.items.forEach(it=>{
      const b = resolveRefBase(it.ref);
      if(!b) return;
      n = addNutrients(n, scaleNutrients(b.nutrients, it.qty));
      grams += (b.servingGrams||0) * it.qty;
    });
    return {name:m.name, servingLabel:'1 recipe', servingGrams:grams, nutrients:n, ...};
  }
  return null;
}
```

There are exactly three kinds of reference: 'db' (one of the 332 built-in foods), 'custom' (a food the user created), and 'meal' (a custom recipe made of several other foods). The 'meal' case is recursive — it calls `resolveRefBase` again on each of its own ingredients — which correctly handles a recipe containing several ingredients, each individually looked up and summed, in one small function.

## 45. app.js Part 4 — The Diary & Meal System

Recall that `STATE.diary` is an object mapping date-key strings to that day's data:

```
STATE.diary = {
  '2024-09-17': {
    'meal-1': [ {ref:{type:'db', id:'rice-cooked-white'}, qty: 1.5}, ... ],
    'meal-2': [ ... ],
    water: 2000,           // ml logged
    workouts: [ {name:'Run', duration:30} ]
  },
  '2024-09-16': { ... }
};
```

```
function getDay(key){
  if(!STATE.diary[key]) STATE.diary[key] = {};
  const d = STATE.diary[key];
  STATE.mealSlots.forEach(s=>{ if(!d[s.id]) d[s.id] = []; });
  if(typeof d.water !== 'number') d.water = 0;
  if(!Array.isArray(d.workouts)) d.workouts = [];
  return d;
}
```

`getDay()` is a "get-or-create" function — a very common pattern. Rather than every caller checking "does today's entry exist yet?", this one function guarantees a fully-formed day object always comes back, creating and filling in any missing pieces (each meal slot's empty array, a zero water total, an empty workout list) on the fly. This also means adding a brand new meal slot later still works for old diary days that were created before that slot existed — `getDay()` patches it in automatically.

```
function dayTotals(key){
  const d = getDay(key);
  let total = zeroNutrients();
  STATE.mealSlots.forEach(s=>{
    (d[s.id]||[]).forEach(entry=>{ total = addNutrients(total, entryNutrients(entry)); });
  });
  return total;
}
```

`dayTotals()` combines everything from Chapter 44: for every meal slot, for every entry logged in it, resolve its nutrients (scaled by quantity) and fold it into a running total with `addNutrients`. This one function powers the calorie ring, the macro bars, and the entire `Nutrients` tab.

```
function computeStreak(){
  let d = todayKey();
  if(!dayHasEntries(d)) d = addDaysKey(d, -1); // don't break the streak just because
                                              // today hasn't been logged YET

  let count = 0;
  while(dayHasEntries(d)){ count++; d = addDaysKey(d, -1); }
  return count;
}
```

The "logging streak" (shown as a flame icon + number) counts consecutive days backward from today that have at least one food logged. The small but thoughtful detail: if *today* has nothing logged yet, it starts counting from yesterday instead — so your streak doesn't visually reset to zero first thing in the morning before you've

had a chance to log breakfast. This is exactly the kind of `while` loop from Part 1's Loops chapter, applied to real data.

## 46. app.js Part 5 — Energy Calculations (BMR/TDEE)

This section implements real nutrition science: the **Mifflin-St Jeor equation**, a widely-used formula for estimating how many calories someone burns per day at rest.

```
function calcBMR(){
  const p = STATE.profile;
  if(!p.sex || !p.age || !p.heightCm) return null; // can't calculate without these
  const weight = latestWeightKg();
  if(weight == null) return null;
  const s = p.sex === 'male' ? 5 : -161;
  return 10*weight + 6.25*p.heightCm - 5*p.age + s;
}
```

BMR (Basal Metabolic Rate) is roughly "calories burned just to stay alive, lying still all day." The formula needs weight (kg), height (cm), age (years), and biological sex (the sole reason this formula needs it: average differences in body composition), each multiplied by its own scientifically-derived coefficient. Note the guard clauses at the top — recall Part 1's Objects/Functions chapters — if any required profile field is missing, the function returns `null` rather than calculating with garbage/missing data, and every caller of this function is expected to check for that `null` before using the result (Chapter 52 shows the UI's fallback message when it's missing).

```
function calcTDEE(){
  const bmr = calcBMR(); if(bmr == null) return null;
  const mult = ACTIVITY_MULT[STATE.profile.activityLevel || 'moderate'];
  return bmr * mult;
}
```

TDEE (Total Daily Energy Expenditure) scales BMR up by an activity multiplier (from `ACTIVITY_MULT`) to account for exercise and daily movement — someone who's very active burns meaningfully more than their BMR alone.

```
function calculateTargetsFromProfile(){
  const bmr = calcBMR();
  if(bmr == null){ toast('Add your sex, age, and height in Profile first.');
```

`return; }`

```
  const weight = latestWeightKg();
  const tdee = calcTDEE();
  const p = STATE.profile;
  const dailyAdj = ((p.weeklyRateKg||0) * 7700) / 7; // 7700 kcal ~ 1 kg of body fat
  let calorieGoal = tdee;
  if(p.goalType === 'cut') calorieGoal = tdee - dailyAdj;
  else if(p.goalType === 'bulk') calorieGoal = tdee + dailyAdj;
  calorieGoal = Math.max(1200, Math.round(calorieGoal)); // never recommend below 1200 kcal
  const proteinPerKg = p.goalType === 'cut' ? 2.0 : 1.8;
  const proteinGoal = Math.round(weight * proteinPerKg);
  const fatGoal = Math.round(calorieGoal*0.25/9); // ~25% of calories from fat (9 kcal/g)
  const carbGoal = Math.max(0, Math.round((calorieGoal - proteinGoal*4 - fatGoal*9)/4)); // remainder as carbs
  ...
}
```

This function turns the science into an actionable set of daily goals with one button click ("Calculate my targets", Chapter 52). It uses the well-known approximation that roughly 7,700 kcal of sustained surplus or deficit corresponds to about 1 kg of body weight change, to translate a user's desired weekly rate of loss/gain into a daily calorie adjustment. It sets protein higher (per kg of bodyweight) during a "cut" (calorie deficit) than

during maintenance or a "bulk," reflecting real nutrition guidance to preserve muscle while losing fat. Fat and carbs are then derived from what's left of the calorie budget, using the standard calorie-per-gram figures (9 kcal/g fat, 4 kcal/g carbs/protein). Note the safety floor: `Math.max(1200, ...)` refuses to ever suggest a dangerously low calorie target, regardless of what the math alone would produce.

## 47. app.js Part 6 — The Rendering System

This is the architectural core of the whole UI, and it's worth understanding deeply, because it's the pattern that replaces what a framework like React would normally provide.

```
function renderApp(){ renderShell(); renderView(); }

function renderShell(){
  document.getElementById('sideNav').innerHTML = `...`; // nav, streak, theme buttons
  document.getElementById('bottomNav').innerHTML = renderNav();
  document.getElementById('topbarTitle').textContent = ({...})[currentView];
}

function renderView(){
  const root = document.getElementById('viewRoot');
  let html = '';
  if(currentView === 'today') html = renderToday();
  else if(currentView === 'foods') html = renderFoodsView();
  else if(currentView === 'progress') html = renderProgressView();
  else if(currentView === 'nutrients') html = renderNutrientsView();
  else if(currentView === 'profile') html = renderProfileView();
  root.innerHTML = `

The pattern, in one sentence: every render function is a pure function that takes the current STATE and returns an HTML string — it reads data, builds a template-literal string full of ${...} interpolations (Part 1's template literals chapter), and hands it back. Nothing inside renderToday(), for instance, ever touches the DOM directly; it just returns text. The caller — renderView() — is the only place that actually assigns to .innerHTML, replacing everything inside #viewRoot in one shot.



### 47.1 The render cycle — how a click becomes a screen update



This five-step loop happens every single time the user does anything at all in this app:



- 1. User clicks/types/submits something.
- 2. The event delegation system (Chapter 55) figures out what action that corresponds to.
- 3. The handler code directly mutates STATE (e.g. pushes a new entry into today's meal array).
- 4. The handler calls saveState() (persists to localStorage) and then renderView() or renderApp().
- 5. The relevant render function reads the now-updated STATE and regenerates the HTML from scratch; the browser updates the screen.



Note: This is a much simpler, hand-rolled version of the same idea frameworks like React are built around: "the UI is a function of state." The tradeoff for not using a real framework is efficiency — Tiffin regenerates and replaces entire chunks of HTML on every change (no fine-grained diffing), which is wasteful in theory, but for an app this size, running on a modern device, it's fast enough to be invisible to the user, and it comes with zero framework dependency and near-zero build tooling. This is a real, defensible trade-off — see Chapter 57.



### 47.2 Lazy-loading Chart.js


```

```
function ensureChartLoaded(cb){
  if(typeof Chart !== 'undefined'){ cb(); return; }           // already loaded, run callback now
  if(chartLoadState === 'loading'){ /* wait and retry */ return; }
  chartLoadState = 'loading';
  const s = document.createElement('script');
  s.src = CHART_CDN_URL;
  s.onload = ()=>{ chartLoadState='loaded'; cb(); };
  s.onerror = ()=>{ chartLoadState='error'; cb(); };
  document.head.appendChild(s);
}
```

Rather than always loading the (fairly large) Chart.js library up front, the app only fetches it the first time the user opens the Progress tab, by dynamically creating a `<script>` element pointed at a CDN URL and inserting it into the page (exactly the DOM-creation pattern from Part 1's DOM chapter). It tracks loading state so simultaneous or repeated requests don't trigger duplicate downloads, and handles the failure case (no internet) gracefully by still calling the callback, which then shows a "chart unavailable" fallback instead of crashing.

## 48. app.js Part 7 — The Today View & Nutrients Tab

`renderToday()` is the single most complex render function, and a good example of composing several smaller render functions together:

```
function renderToday(){
  const totals = dayTotals(currentDate);
  ...
  const macroRows = [
    barRow('Protein', totals.protein, g.proteinGoal, 'g'),
    barRow('Carbs', totals.carbs, g.carbGoal, 'g'),
    barRow('Fat', totals.fat, g.fatGoal, 'g'),
  ].join('');

  const mealSections = STATE.mealSlots.map(slot=>{
    const items = day[slot.id] || [];
    const rows = items.length ? items.map((entry, idx) => `...`).join('') : `<div class="meal-empty">...</div>`;
    return `<div class="meal-card">...${rows}...</div>`;
  }).join('');

  return `
    ${dateNavHtml()}
    <div class="card">
      ${ringSvg(kcalPct, fillColor)}
      <div class="macro-rows">${macroRows}</div>
    </div>
    ${waterCardHtml()}
    ${workoutCardHtml()}
    ${mealSections}
  `;
}
```

Every piece — the date navigator, the calorie ring, the macro bars, the water card, the workout card, each meal card — is its own small function returning its own HTML fragment. `renderToday()`'s only job is to call each one and stitch the results together with template literals. This mirrors "component composition" from frameworks, just without any framework machinery behind it — a function returning a string *is* the component.

### 48.1 The SVG progress ring

```
function ringSvg(pct, fillColor){
  const size=200, cx=size/2, cy=size/2, r=84;
  const circumference = 2*Math.PI*r;
  const p = clamp(pct,0,1);
  const dash = circumference*p;
  const dasharray = circumference*p;
  return `<svg viewBox="0 0 ${size} ${size}" style="transform:rotate(-90deg);">
    <circle cx="${cx}" cy="${cy}" r="${r}" fill="none" stroke="var(--surface-2)" stroke-width="16"/>
    <circle cx="${cx}" cy="${cy}" r="${r}" fill="none" stroke="${fillColor}" stroke-width="16"
      stroke-dasharray="${dash} ${circumference}"/>
  </svg>`;
}
```

This is a classic SVG trick for drawing a progress ring without any image or canvas drawing: draw a full circle twice — once as a plain gray "track", then again on top with a dashed stroke pattern where the first "dash" length equals *percentage complete*  $\times$  *the circle's full circumference* and the second number is the remainder, which SVG then renders as one continuous colored arc covering exactly that percentage of the circle. Rotating the whole SVG -90 degrees makes the arc start from the top (12 o'clock) instead of SVG's default 3 o'clock

starting point.

## 48.2 The Nutrients tab

```
function renderNutrientsView(){
  const totals = dayTotals(currentDate);
  const G = Object.assign({}, FIXED_TARGETS, STATE.goals);
  const groups = NUTRIENT_GROUPS.map(grp=>{
    const rows = grp.rows.map(([key, label, unit, goalKey, isLimit])=>{
      const goal = goalKey ? G[goalKey] : null;
      return barRow(label, totals[key]||0, goal, ' '+unit, {isLimit});
    }).join('');
    return `<div class="nutrient-group-title">${grp.title}</div><div>${rows}</div>`;
  }).join('');
  return `${dateNavHtml()}<div class="card">...${groups}...</div>`;
}
```

This function is a great example of Part 1's Object/Array chapters paying off directly: `Object.assign({}, FIXED_TARGETS, STATE.goals)` merges the app-wide default targets with whatever the user has personally customized (the user's own values winning any overlap), and the array destructuring on each row — `([key, label, unit, goalKey, isLimit])` — unpacks each nutrient's metadata tuple directly in the `.map()` callback's parameter list.

## 49. app.js Part 8 — The Food Picker Flow

Adding a food is a small multi-step "wizard" flow, entirely modeled as one JavaScript object, `picker`, that gets read and mutated as the user moves through it:

```
function openPicker(opts){
  picker = Object.assign({
    step:'list', query:'', cat:'All', meal:null, dateKey:currentDate,
    cart: [], activeRef:null, activeUnit:'medium', activeAmount:1, ...
  }, opts||{});
  renderPickerModal();
}
function renderPickerModal(){
  if(!picker) return;
  const html = picker.step === 'list' ? renderListStep() : renderServingStep();
  openModalHtml(html, 'generic');
}
```

`picker.step` is a simple state machine with two states: `'list'` (search and browse foods) and `'serving'` (pick a quantity for one selected food). `renderPickerModal()` just checks which step it's in and calls the matching render function — the exact same "render function reads state, returns HTML" pattern from Chapter 47, scoped down to a modal instead of a whole page.

The picker also supports a shopping-cart-like flow: tapping a food doesn't immediately log it — it walks you to the serving-size step, then back to the list (with that food now shown in a running "Added so far" cart), so you can add several foods to one meal in a row before finally hitting "Finish" to commit them all to the diary at once:

```
if(action === 'picker-finish'){
  const meal = picker.meal || STATE.mealSlots[0].id;
  const day = getDay(picker.dateKey);
  picker.cart.forEach(item=>{
    const base = resolveRefBase(item.ref);
    const qty = base && base.servingGrams>0 ? item.grams/base.servingGrams : 1;
    day[meal].push({ref:item.ref, qty});
  });
  saveState(); closeModal();
  renderView();
}
```

The same underlying picker object is reused for several related flows by just changing its starting options: `singleMode` (skip the cart, add immediately — used by the Foods tab's "quick add"), editing an existing entry (pre-fill `editMeal/editIndex` so "Add" becomes "Save" and replaces the old entry instead of creating a new one), and `linkOnly` (used when linking a scanned barcode to an existing food, Chapter 53, where picking a food should link it rather than log it).

## 50. app.js Part 9 — Custom Foods & Custom Meals

A custom food form dynamically generates one input field per nutrient, reusing NUTRIENT\_GROUPS so the form's organization exactly matches the Nutrients tab's organization:

```
function customFoodNutrientFieldsHtml(){
  return NUTRIENT_GROUPS.map(grp=>{
    const rows = grp.rows.filter(([key])=> !CUSTOM_FOOD_PRIMARY_KEYS.includes(key));
    ...
    return `<details class="settings-group"><summary>${grp.title}</summary>${pairsHtml}</details>`;
  }).join('');
}
```

(<details></summary> is a native HTML element pair — from Part 1's HTML chapters — that creates a collapsible section with zero JavaScript needed. Calories, protein, carbs, and fat are excluded from this generated list because they're always shown as separate, prominent fields at the top of the form — everything else (all ~40 vitamins/minerals/fat-subtypes) is generated.

```
function handleCustomFoodSubmit(form){
  const fd = new FormData(form);
  const name = (fd.get('name')||'').toString().trim();
  if(!name){ toast('Please name this food.');
```

FormData is a built-in browser API (Part 1's Forms/Browser-APIs chapters) that reads every named input inside a <form> at once, keyed by each input's name attribute — much simpler than manually querying and reading dozens of individual input elements. The small inline arrow function num(k, d) parses a field as a number and falls back to a default d if it's empty or invalid, so a nutrient field left blank quietly becomes 0 rather than NaN corrupting later math.

### 50.1 Custom meals (recipes)

A custom meal is built interactively: search for an ingredient, tap it to add it to the recipe, repeat, then save. Recall that a meal reference resolves recursively (Chapter 44) — so once saved, a custom meal behaves exactly like any other loggable food everywhere else in the app, including being usable as an ingredient in searches, but not nestable inside another custom meal (the picker explicitly filters meal-type items out of the ingredient search to prevent that).

## 51. app.js Part 10 — Progress View & Charts

The weight-trend chart is a thin wrapper around the Chart.js library, lazy-loaded via the mechanism from Chapter 47.2:

```
function renderWeightChartNow(){
  const canvas = document.getElementById('weightChart');
  if(typeof Chart === 'undefined'){ /* show fallback text instead */ return; }
  const log = [...STATE.weightLog].sort((a,b)=> a.date < b.date ? -1 : 1);
  const labels = log.map(w=>w.date.slice(5));
  const data = log.map(w=> round1(toDisplayWeight(w.kg)));
  if(weightChart){ weightChart.destroy(); weightChart = null; } // clean up the old chart first!
  weightChart = new Chart(canvas.getContext('2d'), {
    type:'line',
    data:{ labels, datasets:[ {label:'Weight', data, ...}, ...(goal!=null ? [{label:'Goal', ...}] : []) ] },
    options:{ responsive:true, ... }
  });
}
```

Two details worth knowing for an interview: (1) `weightChart.destroy()` is called before creating a new chart instance — Chart.js attaches its own internal event listeners and canvas state to each chart object, and failing to destroy the old one before replacing it (which happens every time this view re-renders) would leak memory and cause visual glitches over time; and (2) the optional goal-line dataset is added conditionally using the array-spread trick from Part 1's Modern Syntax chapter — `...(goal!=null ? [{...}] : [])` spreads in either one extra dataset object or nothing at all, cleanly avoiding an `if` statement in the middle of an object literal (which isn't syntactically possible in JavaScript).

The chart's colors are read directly from the current CSS variables (via a small `css()` helper) so the chart automatically matches whichever theme — light or dark — is currently active, without any separate "chart theme" configuration.

## 52. app.js Part 11 — Profile View

The Profile view is the largest single render function in the app, but structurally it's just several independent `<form>` elements stacked together — each with its own `id`, handled separately in `onSubmit` (Chapter 55). A few UI details worth noting:

- Height can be entered in cm or ft/in — internally, the app *always* stores height as centimeters (`heightCm`); the ft/in inputs are converted on the way in and out, so all downstream math (like `calcBMR`) only ever has to deal with one unit.
- Similarly, weight is always stored internally in kilograms, with `toDisplayWeight()` / `fromDisplayWeight()` converting to/from pounds only at the point of display or input, based on the user's chosen unit preference.
- The micronutrient goal-editing section is wrapped in a collapsible `<details>` element (Chapter 50) since most users won't want to micromanage 40 individual targets.

Notice the design principle here — introduced back in Part 1's units discussion: pick ONE canonical internal representation (centimeters, kilograms) and convert only at the boundary where a human reads or types a value, rather than letting multiple units leak into the actual math anywhere in the codebase.

## 53. app.js Part 12 — Barcode Scanning

This feature combines three ideas already covered in Part 1: the `BarcodeDetector` and `MediaDevices` browser APIs, `async/await`, and a recursive polling loop using `requestAnimationFrame`:

```
async function startScan(){
  if(!('BarcodeDetector' in window)){ showScanMessage("not supported"); return; }
  try{
    scanDetector = new window.BarcodeDetector({formats});
    scanStream = await navigator.mediaDevices.getUserMedia({video:{facingMode:'environment'}});
    video.srcObject = scanStream;
    await video.play();
    scanning = true;
    scanLoop();
  }catch(err){ showScanMessage("Camera access isn't available here."); }
}

function scanLoop(){
  if(!scanning) return;
  scanDetector.detect(video).then(codes=>{
    if(codes && codes.length){ handleBarcode(codes[0].rawValue); }
    else if(scanning){ requestAnimationFrame(scanLoop); } // check again next frame
  }).catch(()=>{ if(scanning) requestAnimationFrame(scanLoop); });
}
```

`facingMode: 'environment'` requests the phone's rear (outward-facing) camera specifically, since that's the one normally used for scanning things.

When a code is found, `handleBarcode()` looks it up in `STATE.barcodeMap` (Chapter 42); if it's already linked to a food, it offers to log that food directly; if it's a never-before-seen barcode, it offers to either link it to an existing food or create a brand new custom food for it — after which future scans of that same barcode resolve instantly.

## 54. app.js Part 13 — Export / Import / Theme

### 54.1 Backup export

```
async function exportData(){
  const json = JSON.stringify(STATE, null, 2);
  const filename = 'tiffin-backup-'+todayKey()+'.json';
  try{
    if(window.claude && window.claude.use){
      const downloads = await window.claude.use('downloads');
      if(downloads){ await downloads.save({filename, data: json}); return; }
    }
  }catch(e){}
  // fallback: show the JSON in a text box with a "copy" button
  openModalHtml(`<textarea readonly>${escapeHtml(json)}</textarea> ...`, 'generic');
}
```

`JSON.stringify(STATE, null, 2)` — from Part 1's JSON chapter — pretty-prints with 2-space indentation, making the exported file human-readable if someone opens it directly. The function first tries an optional platform-specific save-file API (available when running inside certain host environments), and falls back to simply displaying the raw JSON text in a read-only textbox with a "copy to clipboard" button when that's not available — always giving the user *some* way to get their data out, regardless of environment.

### 54.2 Backup import

```
function importFromFile(file){
  const reader = new FileReader();
  reader.onload = ()=>{
    try{
      const parsed = JSON.parse(reader.result);
      const s = defaultState();
      s.goals = mergeKnownFields(DEFAULT_GOALS, parsed.goals); // <-- security, Ch.43.2
      s.profile = mergeKnownFields(defaultProfile(), parsed.profile); // <-- security, Ch.43.2
      ...
      STATE = s;
      saveState(); applyTheme(); renderApp();
    }catch(e){ toast("That file couldn't be read as a Tiffin backup."); }
  };
  reader.readAsText(file);
}
```

This is exactly the loader logic from `loadState()` (Chapter 43), reused for an imported file — and it's the more important of the two places `mergeKnownFields` gets used, since an imported file is more plausibly something a user downloaded, shared, or hand-edited, rather than something the app wrote itself.

### 54.3 Theme switching

```
function applyTheme(){
  const t = STATE.goals.theme || 'system';
  if(t === 'system') document.documentElement.removeAttribute('data-theme');
  else document.documentElement.setAttribute('data-theme', t);
}
```

This is the JavaScript half of the dark-mode system introduced in Chapter 40.1: setting or removing the `data-theme` attribute on the root `<html>` element is the ONLY thing this function does — every visual consequence is handled entirely by the CSS variable rules already defined in `app.css` reacting to that attribute changing.

## 55. app.js Part 14 — The Event System (the app's heart)

Everything explained so far describes functions that *could* run — this section is what actually decides *when* they run. Recall the event delegation pattern from Part 1 (Chapter 26); this is its full, real implementation.

```
function onClick(e){
  const t = e.target.closest('[data-action]');
  if(!t) return;
  const action = t.dataset.action;

  if(action === 'nav'){ currentView = t.dataset.view; renderApp(); return; }
  if(action === 'date-prev'){ currentDate = addDaysKey(currentDate,-1); renderView(); return; }
  if(action === 'open-picker'){ openPicker({meal:t.dataset.meal}); return; }
  if(action === 'delete-entry'){
    e.stopPropagation();
    const meal = t.dataset.meal, idx = parseInt(t.dataset.index,10);
    getDay(currentDate)[meal].splice(idx,1);
    saveState(); renderView();
    return;
  }
  // ... roughly 60 more branches, covering every button in the app
}
```

This single function is around 260 lines long and handles essentially every click in the entire application — nav switching, date navigation, opening every modal, every delete button, every settings toggle, the whole food picker flow, barcode scanning controls, meal slot management, and more. Each branch follows an almost identical shape: read some data out of the clicked element's dataset (Part 1's DOM chapter), mutate STATE or another module-level variable accordingly, then return immediately — the early return after every branch is what makes an otherwise 60-branch if-chain readable: once one matches, nothing below it is even checked.

`e.stopPropagation()`, seen in the delete-entry branch, prevents this click from also bubbling up and triggering a *different* data-action on a parent element (in this specific case, the delete (×) icon sits inside a row that itself has its own data-action="edit-entry" for opening the edit screen — without `stopPropagation()`, clicking delete would also immediately trigger edit — exactly the distinction Part 1's Events chapter drew between `stopPropagation` and `preventDefault`).

### 55.1 Three sibling listeners for three other event types

```
function onInput(e){    // fires on every keystroke (live search-as-you-type, live nutrient preview)
  const el = e.target;
  if(el.id === 'foodsSearch'){ foodsFilter.query = el.value; /* re-render just the list */ return; }
  if(el.id === 'servingAmount'){ /* recompute and redraw just the nutrition preview numbers */ return; }
  ...
}
function onChange(e){  // fires when a <input type="file"> selection is confirmed
  if(el.id === 'importFileInput' && el.files[0]){ importFromFile(el.files[0]); return; }
}
function onSubmit(e){  // fires when any <form> is submitted
  if(form.id === 'customFoodForm'){ e.preventDefault(); handleCustomFoodSubmit(form); return; }
  if(form.id === 'profileForm'){ e.preventDefault(); /* save name */ return; }
  if(form.id === 'weightForm'){ e.preventDefault(); /* validate + save a weigh-in */ return; }
  ...
}
```

`e.preventDefault()` inside every form handler stops the browser's default behavior for a form submission — which is to reload the entire page and send the data to a server via a URL, exactly what a no-backend app must never let happen (recall Part 1's forms chapter). Note also that `onInput`'s live-search handler deliberately updates *only* the small results-list element (`document.getElementById('foodsList').innerHTML = ...`) rather than calling the full `renderView()` — re-rendering the entire page on every single keystroke of a search box would be needlessly wasteful and could even steal focus away from the input the user is actively typing in.

## 56. app.js Part 15 — Startup

```
let initied = false;
function init(){
  if(initied) return; initied = true;
  applyTheme();
  document.addEventListener('click', onClick);
  document.addEventListener('input', onInput);
  document.addEventListener('change', onChange);
  document.addEventListener('submit', onSubmit);
  document.getElementById('modalOverlay').addEventListener('click', function(e){
    if(e.target.id === 'modalOverlay') closeModal(); // click OUTSIDE the modal box closes it
  });
  renderApp(); // the very first paint of the whole app
}
document.addEventListener('DOMContentLoaded', init);
if(document.readyState !== 'loading') init();
```

This is the last thing in the file, and it's where everything described in this entire part actually kicks off: apply the saved theme, attach the four delegated event listeners from Chapter 55 exactly once, wire up "click the dark overlay behind a modal to close it," and finally call `renderApp()` for the very first time, painting the initial screen from whatever STATE was loaded at the very top of the file (Chapter 43).

The `initied` guard flag exists because of the two lines at the very bottom: `DOMContentLoaded` (Part 1's Events chapter) fires once, when the browser finishes parsing the HTML — but if the script happens to load and execute *after* that event already fired (a real possibility depending on exactly how/when the script tag runs), the listener would simply never fire at all, and the app would never start. The second line, checking `document.readyState`, catches that case by calling `init()` immediately if the document has already finished loading. The guard flag ensures that if *both* paths somehow end up running, `init()` still only ever truly executes once.

# **PART 3**

## **Architecture & Design Decisions**

## 57. Why No Framework, No Backend?

### 57.1 No framework (no React / Vue / Angular)

This app achieves everything a framework normally provides — a render-on-state-change loop, reusable "components," event handling — using nothing but the patterns from Chapters 47 and 55: template literals for components, one delegated listener for events, and full-innerHTML-replacement for updates.

- **Pro:** Zero dependencies, zero build step needed to develop (only to bundle for shipping), and a single ~1,700-line file that a reader can genuinely hold in their head — nothing hides behind an abstraction layer or framework "magic."
- **Con:** No fine-grained DOM diffing (a framework only updates the exact DOM nodes that actually changed; this app replaces whole chunks of HTML every time), no built-in component reuse/composition tooling, and the burden of avoiding bugs like memory leaks (Chapter 51's chart cleanup) falls entirely on the developer instead of the framework.
- **Why it's a reasonable choice here:** the app's total data size is small (one user's diary, not millions of rows), so the performance cost of "just re-render everything" never becomes noticeable, and the simplicity payoff — no build tooling required to even run the app locally, trivial to audit for security since every line of logic is visible in one file — is worth it for a project this size.

### 57.2 No backend, no server, no database

Every user's data lives only in their own browser's localStorage (Part 1, Chapter 30). There is no login, no account, no API calls that send personal data anywhere.

- **Pro:** Perfect privacy by construction (nothing to breach — there's no server holding anyone's data), zero server hosting cost, and the app works fully offline once loaded, since even the food database is baked directly into the page (Chapter 41).
- **Con:** No cross-device sync (your diary on your phone and your laptop are two separate, unconnected datasets), and clearing browser data destroys everything unless a backup was manually exported (Chapter 54).
- **Why it's a reasonable choice here:** a personal nutrition tracker's core value doesn't strictly require sync or accounts, and trading that away buys meaningful simplicity and privacy.

## 58. Security: XSS, CSP, Prototype Pollution

### 58.1 XSS (Cross-Site Scripting) and escapeHtml

Because so much of this app's HTML is built by directly interpolating data into template literals (``<div>${food.name}</div>``), there's a real risk: if `food.name` ever contained something like `<img src=x onerror="steal_data()">` — for instance, typed in by the user themselves when naming a custom food — and it were inserted unescaped, the browser would parse it as real HTML and execute that injected script. This class of vulnerability is called **XSS (Cross-Site Scripting)**.

```
function escapeHtml(s){
  return String(s).replace(/<>"/g, c=>({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'
  }[c]));
}

// used everywhere untrusted text is inserted into HTML:
`<div class="fname">${escapeHtml(food.name)}</div>`
```

`escapeHtml()` — the exact regex from Part 1's Strings & Regex chapter — converts the characters that give HTML its special meaning (`<` `>` `&` `"` `'`) into their harmless text equivalents ("HTML entities"), so `<img>` becomes the literal, inert text `&lt;img&gt;`; instead of being parsed as a tag. This function is called on essentially every single piece of user-controlled or data-sourced text before it's woven into an HTML string anywhere in the app — food names, profile name, custom meal names, barcode numbers, all of it.

### 58.2 The Content-Security-Policy, revisited

`escapeHtml` is the app's *first* line of defense — but defensive engineering means assuming a bug slips through anyway (a future edit that forgets to call it, say). Recall from Chapter 38.1 that `build.py` computes cryptographic hashes of the app's own two legitimate inline scripts and locks the browser down to *only* executing scripts matching those exact hashes. Even in the worst case — an attacker somehow gets an unescaped, malicious string containing a `<script>` tag into the page — the browser's own CSP enforcement would refuse to execute it, because its content wouldn't match either trusted hash. This is called **defense in depth**: multiple independent layers of protection, so that one layer failing doesn't mean total compromise.

### 58.3 Prototype pollution, revisited

Covered in full in Chapter 43.2 — `mergeKnownFields()` ensures that loading saved state or an imported backup file can never inject unexpected properties (like `__proto__`) into the app's data, only ever the specific fields the app already knows about and expects.

## 59. The Full Data Flow, End to End

Putting every earlier chapter together, here is literally everything that happens, in order, when a user adds "1.5 cups of rice" to their lunch:

- **1.** User taps + Add Lunch → the click bubbles to document → `onClick` (Ch.55) matches `data-action="open-picker"` → calls `openPicker({meal: 'meal-2'})` (Ch.49).
- **2.** `openPicker` creates the picker state object and calls `renderPickerModal()`, which calls `renderListStep()` → returns an HTML string → `openModalHtml()` sets `#modalContent.innerHTML` and reveals the modal overlay.
- **3.** User types "rice" → fires an input event → `onInput` (Ch.55.1) updates `picker.query`, calls `filterFoods()` against `FOOD_DB`, and replaces just the results list's `innerHTML`.
- **4.** User taps the "Rice, cooked" result → `onClick` matches `select-food` → sets `picker.activeRef` and `picker.step = 'serving'` → re-renders the modal, now showing `renderServingStep()`'s quantity screen.
- **5.** User sets amount to 1.5 → `onInput` recomputes grams/calories live via `computeGrams()` and `scaleNutrients()` (Ch.44) and updates just the preview numbers on screen.
- **6.** User taps Add to list → `onClick` matches `add-to-cart` → pushes the item into `picker.cart`, returns to the list step.
- **7.** User taps Finish → `onClick` matches `picker-finish` → for each cart item, computes its quantity and `getDay(dateKey)[meal].push({ref, qty})` — this is the moment STATE itself actually changes (Ch.45).
- **8.** Still inside that handler: `saveState()` serializes the whole STATE object to JSON and writes it to `localStorage` (Ch.43) — the data is now durable, surviving even if the tab is closed immediately.
- **9.** `closeModal()` hides the modal; `renderView()` (Ch.47) calls `renderToday()` (Ch.48) fresh — it calls `dayTotals()` (Ch.45), which now includes the new rice entry, recomputes the calorie ring's percentage, and returns brand-new HTML reflecting the updated total.
- **10.** That HTML replaces `#viewRoot`'s content; the browser repaints; the user sees the ring, the macro bars, and the lunch card all instantly reflect the new food — done.

**Note:** Every one of those ten steps is just a composition of small, individually simple functions covered earlier in this guide. There's no framework, no network request, no hidden magic anywhere in this chain — which is precisely the point being made in Chapter 57.

# PART 4

## Interview Guide

You now understand every language this app uses and every significant piece of its code. This part turns that understanding into interview-ready answers. Read the question, try to answer it yourself first, then compare against the model answer.

## 60. The 30-Second Pitch

*"Tiffin is a full-featured nutrition tracker — think Cronometer or MyFitnessPal — built as a single, self-contained, zero-dependency web app in vanilla JavaScript. It tracks 44 nutrients across a 332-food built-in database with an Indian-and-international focus, supports barcode scanning, custom foods and recipes, weight tracking with charts, and BMR/TDEE-based goal calculation using the Mifflin-St Jeor formula. There's no backend and no account system — everything is stored client-side in localStorage, which means it's completely private and works offline. It ships as a website, and also as installable native iOS and Android apps via Capacitor, with both builds automated end-to-end through GitHub Actions."*

## 61. Core Technical Q&A;

### Q: Why vanilla JavaScript instead of React or another framework?

A: For a single-user, client-only app of this size, a framework's main benefits — efficient re-rendering of huge, frequently-changing datasets, and managing complex component trees — aren't really needed. The tradeoff favored simplicity: zero build tooling required to run the app locally, one file that's fully auditable top to bottom, and no dependency updates to manage. The app still uses the same core idea frameworks are built on — 'the UI is a pure function of state' — just implemented by hand with template literals and full innerHTML replacement instead of a virtual DOM.

### Q: How does the app persist data without a server?

A: It uses the browser's localStorage API, which is a simple key-value store that survives page reloads and browser restarts. The app's entire state — profile, goals, diary, custom foods, weight log — is one JavaScript object, serialized to a JSON string with `JSON.stringify` and written under one key. On load, it's read back with `localStorage.getItem` and parsed with `JSON.parse`. This means all data stays on the user's own device and never leaves it.

### Q: Walk me through what happens end to end when a user logs a food.

A: See Chapter 59 for the full ten-step trace — in short: a click is caught by one delegated event listener, which mutates the central STATE object directly, immediately persists it to localStorage, then calls the render function for the current view, which reads the now-updated STATE and returns fresh HTML that replaces what's on screen.

### Q: Explain the event delegation pattern used here, and why.

A: Instead of attaching a separate click listener to every button (which could be hundreds across all the screens), the app attaches exactly one listener to `document`. It relies on event bubbling — a click always propagates up through all of its ancestor elements — so a single top-level handler can inspect `event.target.closest('[data-action]')` to find out what was actually clicked, then dispatch based on that element's `data-action` attribute. This scales to any number of dynamically-created buttons with zero extra listener setup, which matters a lot here since almost the entire UI is regenerated from scratch on every state change.

### Q: What security measures does this app take, and why?

A: Three layers: (1) `escapeHtml()` sanitizes any data-derived text before it's inserted into an HTML template, preventing XSS if a food name or similar field ever contained malicious markup; (2) a strict Content-Security-Policy, computed at build time from SHA-256 hashes of the app's own two legitimate inline scripts, means the browser will refuse to execute any script that doesn't match — a defense-in-depth backstop even if an escaping bug slipped through; (3) `mergeKnownFields()` guards against prototype pollution (CWE-1321) when loading saved or imported JSON data, by only ever copying keys the app already expects, rather than blindly merging an untrusted object's keys, which could otherwise let a key like `'__proto__'` corrupt the behavior of every object in the running app.

### Q: How does the app support both light and dark mode?

A: Through CSS custom properties (variables). The entire color palette is defined once as variables under `:root`, and every other CSS rule references them via `var(--name)` instead of hardcoding colors. A dark-mode block simply redefines the same variable names with dark values, either automatically (via the

prefers-color-scheme media query, following the OS setting) or explicitly (via a data-theme attribute the app sets on based on the user's in-app choice). No individual CSS rule needs to know or branch on which theme is active.

**Q: How is the mobile app (iOS/Android) built from the same code as the website?**

A: Capacitor wraps the built web app (a single static HTML file produced by build.py) inside a native app shell for each platform — essentially a full-screen native WebView pointed at that file. The same JavaScript that runs in a desktop browser tab runs unmodified inside it. Two GitHub Actions workflows automate the whole pipeline on every push: build the web app, run `cap sync` to copy it into the native project, then run each platform's own native compiler (Gradle for Android, Xcode for iOS) to produce an installable .apk or .ipa file as a downloadable build artifact.

**Q: Why does the app store references to foods (type + id) in the diary instead of copies of the nutrition data?**

A: Two reasons: it keeps the diary lightweight, and — more importantly — it means editing a custom food's nutrition values later automatically updates every past diary entry that referenced it, since the nutrients are looked up fresh (resolveRefBase) every time totals are computed, rather than being frozen at the moment they were logged.

**Q: What's a tradeoff of the 'replace the whole innerHTML on every change' rendering approach?**

A: It's simple and easy to reason about, but it's not fine-grained — even a change affecting one small part of the screen regenerates and replaces the whole view's HTML. For an app of this size and a single user's dataset, that's imperceptibly fast on modern devices, so it's a reasonable trade for the simplicity gained. A framework with a virtual DOM would only touch the specific DOM nodes that actually changed, which matters more at larger scale.

## 62. Deep-Dive Design Questions

**Q: Why is the food data (NUTRIENT\_KEYS) represented as a list of strings that other code loops over, instead of just writing out each nutrient calculation by hand?**

A: It turns what would be ~44 nearly-identical lines of code (per function) into one small loop, in every function that touches nutrients: zeroing them out, scaling by quantity, adding two nutrient sets together. It also means the same list drives the Nutrients tab's display grouping and the custom-food form's generated fields — add a 45th nutrient to that one list, and (with matching data in `food_db.json`) it flows through the whole app automatically, rather than needing edits in a dozen different places.

**Q: Why does `calcBMR()` return null instead of, say, 0 or throwing an error, when profile data is missing?**

A: Returning null makes 'no answer is available yet' an explicit, checkable state, distinct from a legitimate calculated value (0 would be misleading — nobody's BMR is actually zero). Every caller checks ``if (bmr == null)`` and shows an appropriate fallback (like a prompt to fill in Profile) instead of either crashing or silently displaying a wrong number.

**Q: Why build a custom HTML file with 4 placeholder substitutions in Python, instead of using a real bundler like Webpack or Vite?**

A: The whole app deliberately has no npm-based build dependencies for its JavaScript — it's plain, unbundled, unminified JS designed to be read and audited directly. A real bundler would be overkill for combining exactly four static files into one; the ~85-line Python script does that one job directly, including computing the CSP script hashes a general-purpose bundler wouldn't know to do out of the box, with no extra dependencies of its own.

**Q: What would you change if this app needed to support multiple devices for the same user?**

A: The core blocker is that `localStorage` is local to one browser on one device — you'd need some remote storage a user's various devices could all read and write, which means introducing a backend and some form of authentication, likely a real API rather than the current 'no server' model. You'd also need a strategy for merging changes made offline on two devices (a sync/conflict-resolution problem), which the current single-device, single-writer model avoids entirely.

**Q: How would you test this codebase?**

A: The pure, render-function-per-view architecture (Ch.47) is actually quite testable in principle: most render functions are pure functions of STATE that return a string, so you could feed them a known STATE object and assert on the returned HTML string without needing a real browser. The nutrient math functions (`scaleNutrients`, `addNutrients`, `dayTotals`) are pure and trivially unit-testable. The harder parts to test are the event-delegation handlers, since they mix DOM reads, STATE mutation, and re-rendering together — those would benefit most from a browser-based testing tool (like Playwright) driving real clicks and asserting on the resulting DOM, since they're what's not currently covered.

## 63. Glossary — Every Term Used in This Guide

**API** — A set of functions/rules that let one piece of software talk to another.

**Array** — An ordered list of values, e.g. [1, 2, 3].

**Async/await** — JavaScript syntax for writing code that waits on slow operations without freezing the page.

**Attribute** — Extra information on an HTML tag, written as name="value".

**Block scope** — Variables (const/let) visible only within the { } block they're declared in.

**Bundler** — A build tool (e.g. Webpack, Vite) that combines many source files into fewer files for shipping.

**CDN** — Content Delivery Network — a server that hosts common public files (like libraries) for fast loading.

**CI/CD** — Continuous Integration/Deployment — automatically building/testing code on every change.

**CSP** — Content-Security-Policy — a browser rule set restricting what scripts/styles are allowed to run.

**CSS** — Cascading Style Sheets — the language that styles HTML's appearance.

**CWE** — Common Weakness Enumeration — a standard catalog of software vulnerability types.

**Callback** — A function passed into another function, to be called later.

**Closure** — A function that remembers variables from the scope it was defined in, even after that scope ends.

**Commit** — A saved snapshot of a project's files in Git, with a description of what changed.

**Component** — A reusable, self-contained piece of UI (here: a function returning an HTML string).

**DOM** — Document Object Model — the browser's live, in-memory tree of a page's elements.

**Data attribute** — A custom HTML attribute starting with data-, readable by JavaScript.

**Destructuring** — JS syntax for unpacking values from an array/object into separate variables.

**Event** — Something that happens on a page: a click, keystroke, form submit, etc.

**Event bubbling** — A DOM behavior where an event fires on an element, then its parent, then its parent's parent, and so on.

**Event delegation** — Listening for events on one shared ancestor instead of each individual element.

**Flexbox** — A CSS layout system for arranging elements in a single row or column.

**Function** — A named, reusable block of code.

**Grid** — A CSS layout system for arranging elements in rows and columns at once.

**HTML** — HyperText Markup Language — describes a page's structure using tags.

**Hoisting** — JavaScript's behavior of making function/variable declarations usable before the line that defines them.

**IIFE** — Immediately Invoked Function Expression — a function defined and called in one step, to create an isolated scope.

**JSON** — JavaScript Object Notation — a text format for structured data.

**Object** — A collection of named key-value pairs, e.g. {name: 'Alice'}.

**Prototype pollution** — A vulnerability class (CWE-1321) where untrusted data corrupts a shared object prototype.

**Regex** — Regular expression — a pattern language for matching text.

**Render** — To convert data into HTML/UI and display it.

**SHA-256** — A cryptographic function that produces a unique fingerprint (hash) of some data.

**Scope** — The region of code from which a given variable is visible/accessible.

**Selector** — A CSS pattern (like .class or #id) that picks which elements a rule applies to.

**Single Page App (SPA)** — A web app that updates its content dynamically in place instead of loading new full HTML pages.

**State** — The current data driving what an app displays.

**String** — Text data, e.g. "hello".

**Template literal** — A backtick-quoted JS string that can embed expressions via \${...}.

**Ternary operator** — A compact if/else expression: condition ? valueIfTrue : valueIfFalse.

**Truthy/falsy** — JavaScript's rules for treating non-boolean values as true or false in a condition.

**Web API** — A built-in browser capability beyond basic HTML/CSS/JS, e.g. camera access or clipboard access.

**XSS** — Cross-Site Scripting — a vulnerability where untrusted text is executed as HTML/script.

**YAML** — A simple, indentation-based text format often used for configuration files.

**const** — A JavaScript variable declaration that can't be reassigned.

**f-string** — Python's syntax for embedding variables directly in a string, e.g. f"Hi {name}".

**let** — A JavaScript variable declaration that CAN be reassigned later.

**localStorage** — A browser API for storing key-value data that persists across sessions.

**Prepared:** September 18, 2026 | **Project:** Tiffin — Nutrition Tracker | **Stack:** HTML, CSS, JavaScript, JSON, Python (build), Capacitor (mobile), GitHub Actions (CI/CD)